

Deep Learning

2026. 7

Yongjin Jeong, KwangWoon University

[참고] 본 자료에는 인터넷에서 다운받아 사용한 그림이나 수식들이 일부
있으니 다른 용도로 사용하거나 외부로 유출을 금해 주시기 바랍니다.

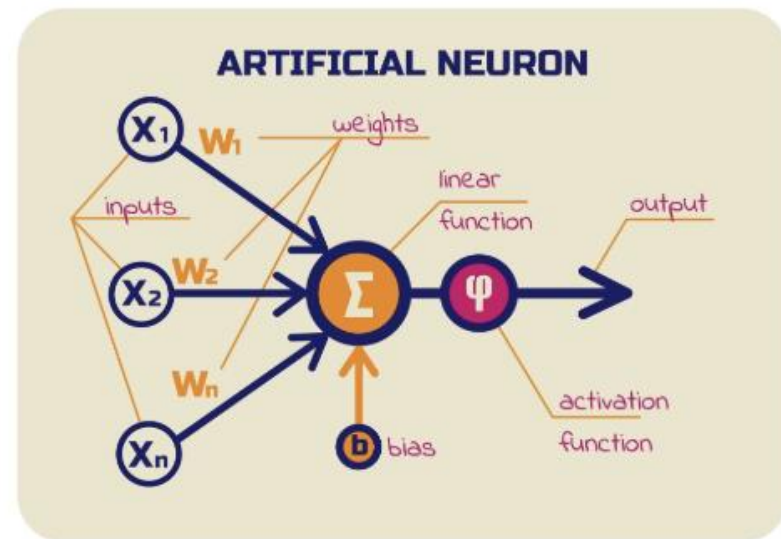
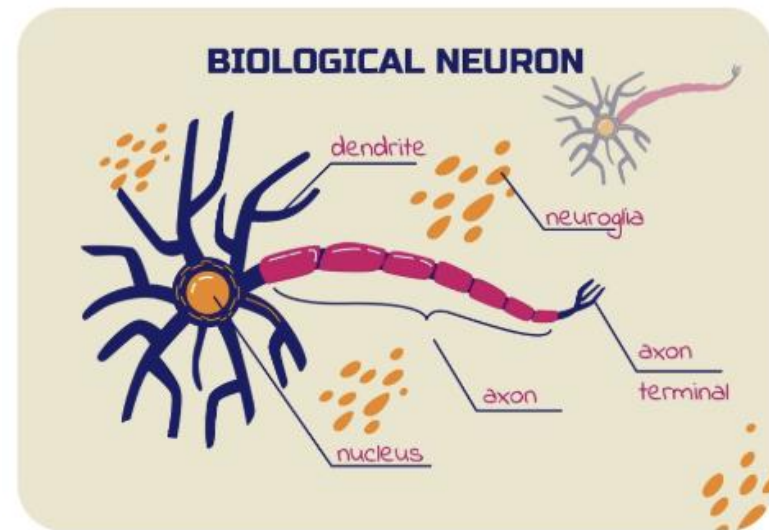
Introduction to Deep Learning

(딥러닝 소개)

딥러닝의 영감, 뇌 신경망

- 뇌 신경망 (Neural Network)

- 딥러닝(deep Learning)은 인간의 뇌를 구성하는 뉴런 (Neuron)의 연결 구조에서 영감을 얻은 인공지능 기술의 한 분야.
- 생물학적 뉴런은 다른 뉴런으로부터 수상돌기(dendrite)를 통해 신호를 받아 세포체(soma)에서 처리한 후, 축삭(axon)을 통해 다음 뉴런으로 신호를 전달한다.
- 이러한 구조를 수학적으로 모방하여 입력(inputs)에 가중치(weights)를 곱하고, 그 합을 활성화 함수(activation function)에 통과시켜 출력(output)을 내는 **인공 뉴런 (Artificial Neuron)** 모델을 사용.



적용 사례: 알파고 (AlphaGo)

- 적용 사례 1 - 알파고 (AlphaGo)

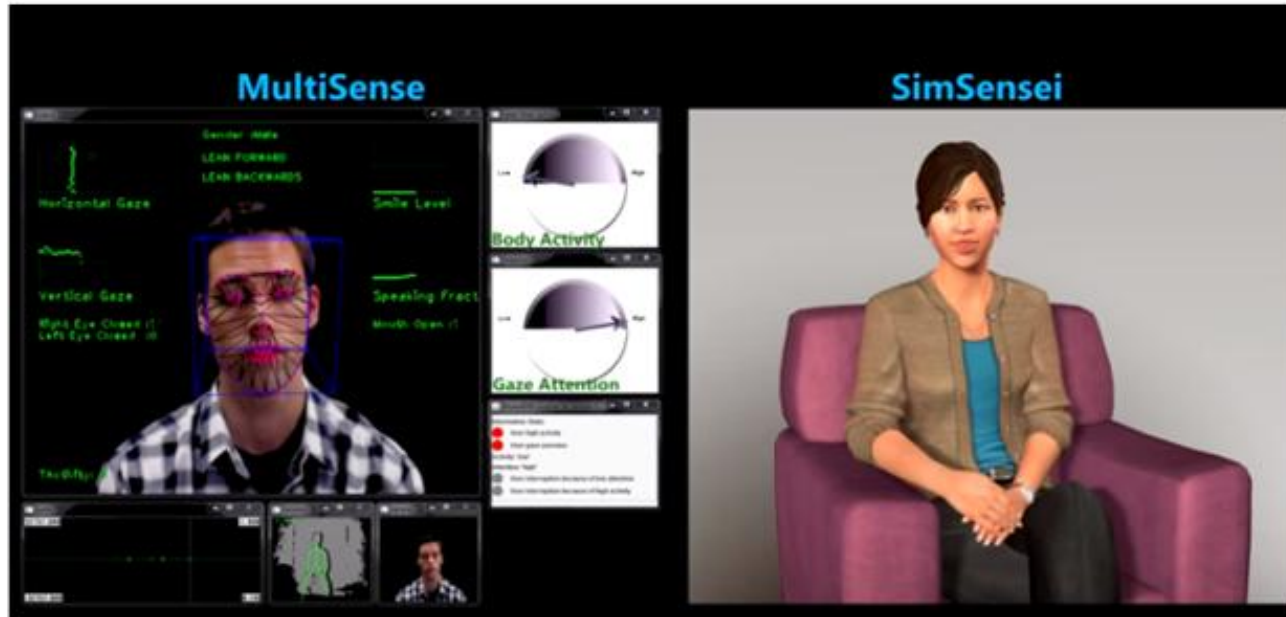
- 2016년, 구글 딥마인드(Google DeepMind)가 개발한 딥러닝 기반 바둑 프로그램 **알파고**가 이세돌 9단과의 대국에서 4승 1패로 승리하며 전 세계에 충격을 주었음.
- 알파고는 기계학습과 병렬처리 기술을 활용했으며, 수십 개의 TPU(Tensor Processing Unit)를 사용한 분산 컴퓨팅 시스템으로 구현되었다. 이후 단 4개의 TPU를 사용하는 AlphaGo Zero로 발전하며 더 뛰어난 성능을 보였다.



적용 사례: SimSensei (감성 컴퓨팅)

- 적용 사례 2 - SimSensei (감성 컴퓨팅)

- 미국 USC 에서 개발한 **SimSensei**는 딥러닝의 안면인식 및 동작인식 기술을 활용해 사람의 우울증 징후를 감지.
- 가상 아바타가 사용자와 대화하는 동안, 소프트웨어는 사용자의 얼굴 근육, 음성 패턴, 자세 등 66개의 특징을 분석하여 기존의 설문지 방식보다 더 정확하게 심리 상태를 파악.



적용 사례: MNIST (필기체 숫자 인식)

- 적용 사례 3 - MNIST (필기체 숫자 인식)

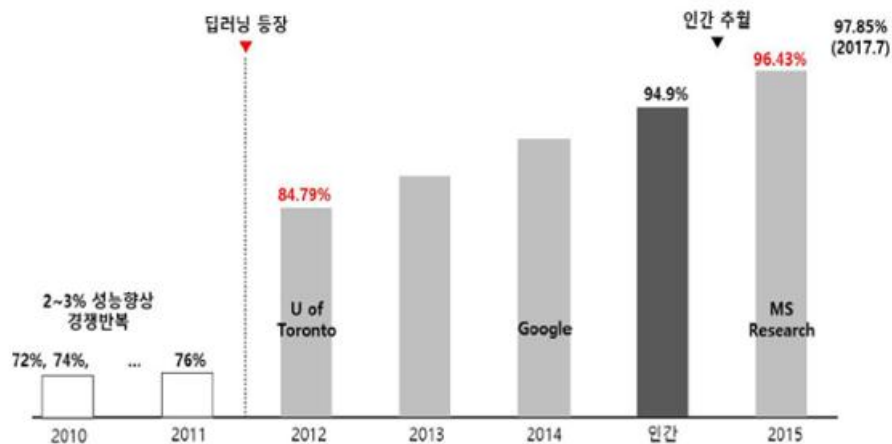
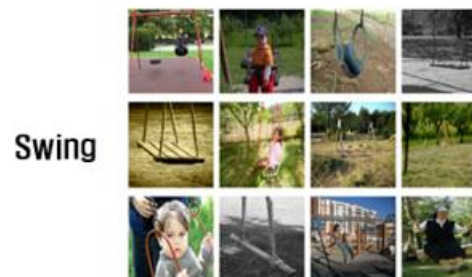
- **MNIST**는 손으로 쓴 숫자 이미지 데이터 집합으로, 딥러닝 모델의 성능을 측정하는 대표적인 기준(benchmark)으로 사용됨.
- 2012년, 딥러닝 기술을 통해 필기체 숫자 인식 오류율이 0.23%까지 떨어지면서, 복잡한 패턴 인식 분야에서 딥러닝의 뛰어난 성능이 입증.



적용 사례: ImageNet Challenge (이미지 인식)

• 적용 사례 4 - ImageNet Challenge (이미지 인식)

- ImageNet 경진대회(ILSVRC)는 1000개의 카테고리 이미지로 이미지를 분류하는 세계 최대 규모의 이미지 인식 대회.
- 2012년, 딥러닝 모델이 처음 등장하여 오류율을 획기적으로 낮추면서 이미지 인식 분야의 판도를 바꾸게 됨.
- 이후 딥러닝 기술은 발전을 거듭하여 2015년에는 인간의 인식 능력을 뛰어넘는 정확도를 달성.

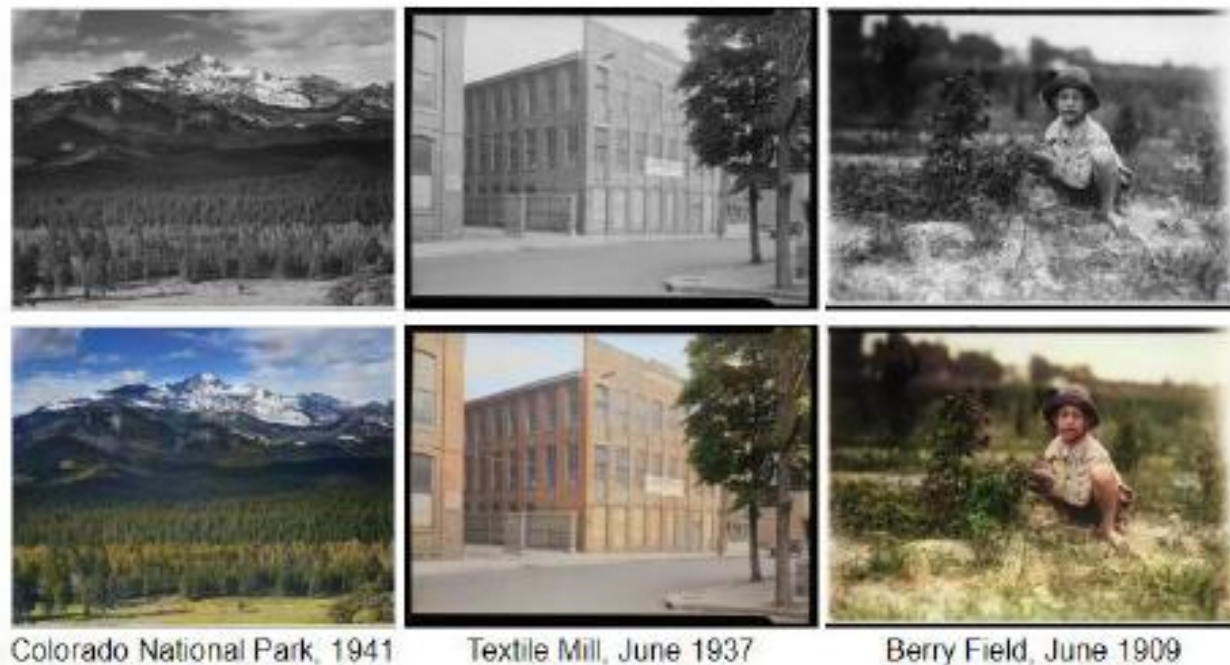
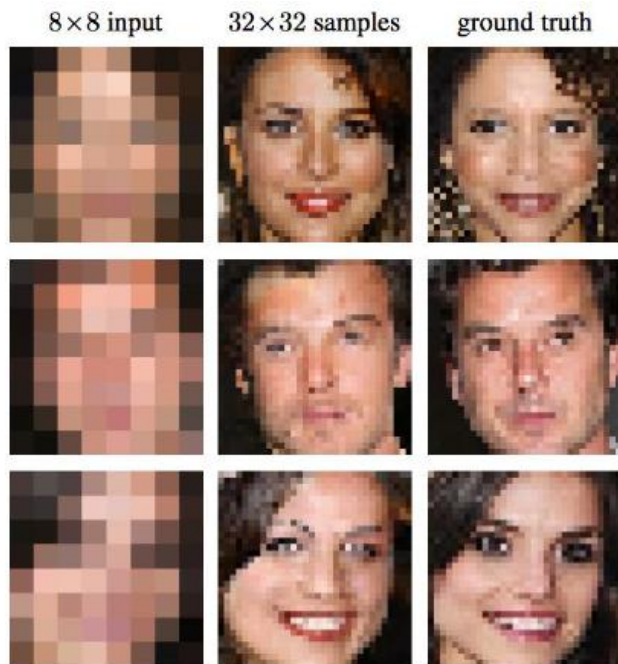


<자료> LG경제연구원, 2017. 10.

적용 사례: 이미지 복원 및 생성

• 적용 사례 5 - 이미지 복원 및 생성

- 손상되거나 정보가 부족한 이미지를 복원하는 데에도 탁월한 능력.
- 이미지 화소 복원(Image Pixel Recovery): 저해상도(8×8)의 얼굴 이미지를 고해상도(32×32) 복원.
- 이미지 색상 복원(Image Color Recovery): 흑백 사진을 보고 원래의 색을 추정하여 자동 채색.



적용 사례: 이미지 캡셔닝 (Image Captioning)

- 적용 사례 6 - 이미지 캡셔닝 (Image Captioning)

- 이미지 캡셔닝: 이미지의 내용을 이해하고, 그 내용을 자연어 문장으로 설명하는 기술.
- "a man throwing a frisbee in a green field." (녹색 들판에서 프리스비를 던지는 남자) 와 같이 사진 속 객체와 상황을 정확히 묘사하는 문장을 자동으로 생성.



a man wearing a blue shirt with his arms on the grass.
a man holding a frisbee bat in front of a green field.
a man throwing a frisbee in a green field.
a boy playing ball with a disc in a field.
a young man playing in the grass with a green ball.



a red car on the side of the road in the small race,
a truck driving uphill on the side of the road.
a person driving a truck on the road.
a small car driving down a dirt and water.
a truck in a field of car is pulled up to the back.



a group of birds standing next to each other,
a group of ducks that are standing in a row,
a group of ducks that are standing on each other,
a group of sheep next to each other on sand.
a group of small birds is standing in the grass.

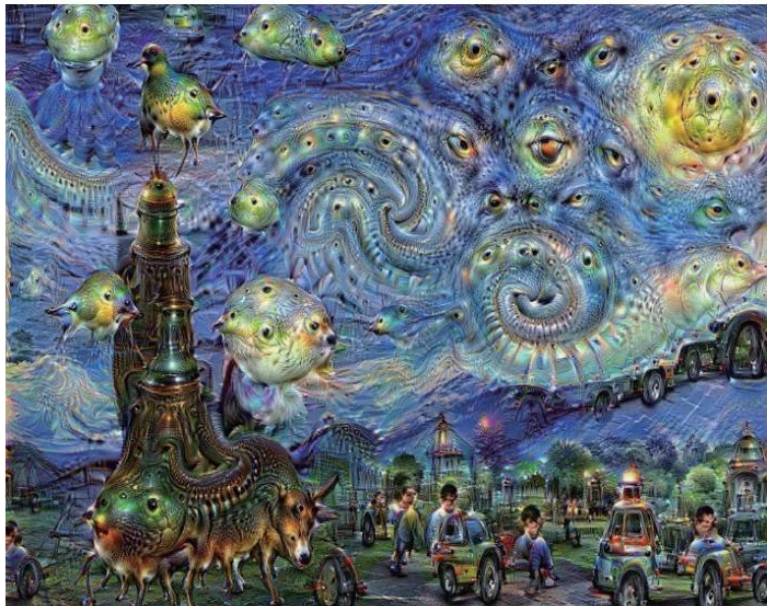


a kite flying over the ocean on a sunny day,
a person flying over the ocean on a sunny day,
a person flying over the ocean on a cloudy day,
a kite on the beach on the water in the sky.
a large flying over the water and rocks.

적용 사례: 딥드림 (Deep Dream)과 예술 창작

- 적용 사례 7 - 딥드림 (Deep Dream)과 예술 창작

- 구글의 딥드림(Deep Dream): 딥러닝 모델이 이미지에서 특정 패턴을 반복적으로 찾아 강화하면서 초현실적이고 몽환적인 이미지를 만들어내는 기술.
- 딥러닝으로 생성된 예술 작품이 경매에서 높은 가격에 판매되는 등, 딥러닝은 이제 창작의 영역까지 확장.



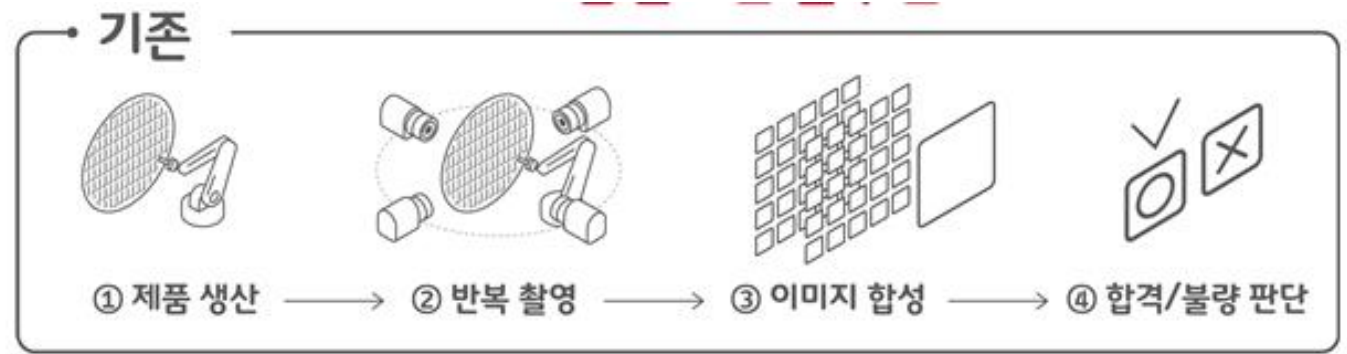
구글 딥드림의 최고가 8,000달러에 낙찰된 작품 ©월스트리트저널



적용 사례: 제조업 혁신 (SKT 슈퍼노바)

• 적용 사례 8 - 제조업 혁신 (SKT 슈퍼노바)

- SK텔레콤의 AI 품질개선 솔루션 '슈퍼노바(Supernova)'는 딥러닝을 반도체 제조 공정에 적용한 사례.
- 기존에는 여러 번 반복 촬영하고 이미지를 합성하여 불량을 판별했지만, 슈퍼노바는 **단 1회 촬영** 후 딥러닝 알고리즘을 통해 합격/불량을 판단하여 작업 속도와 정확도를 대폭 향상.



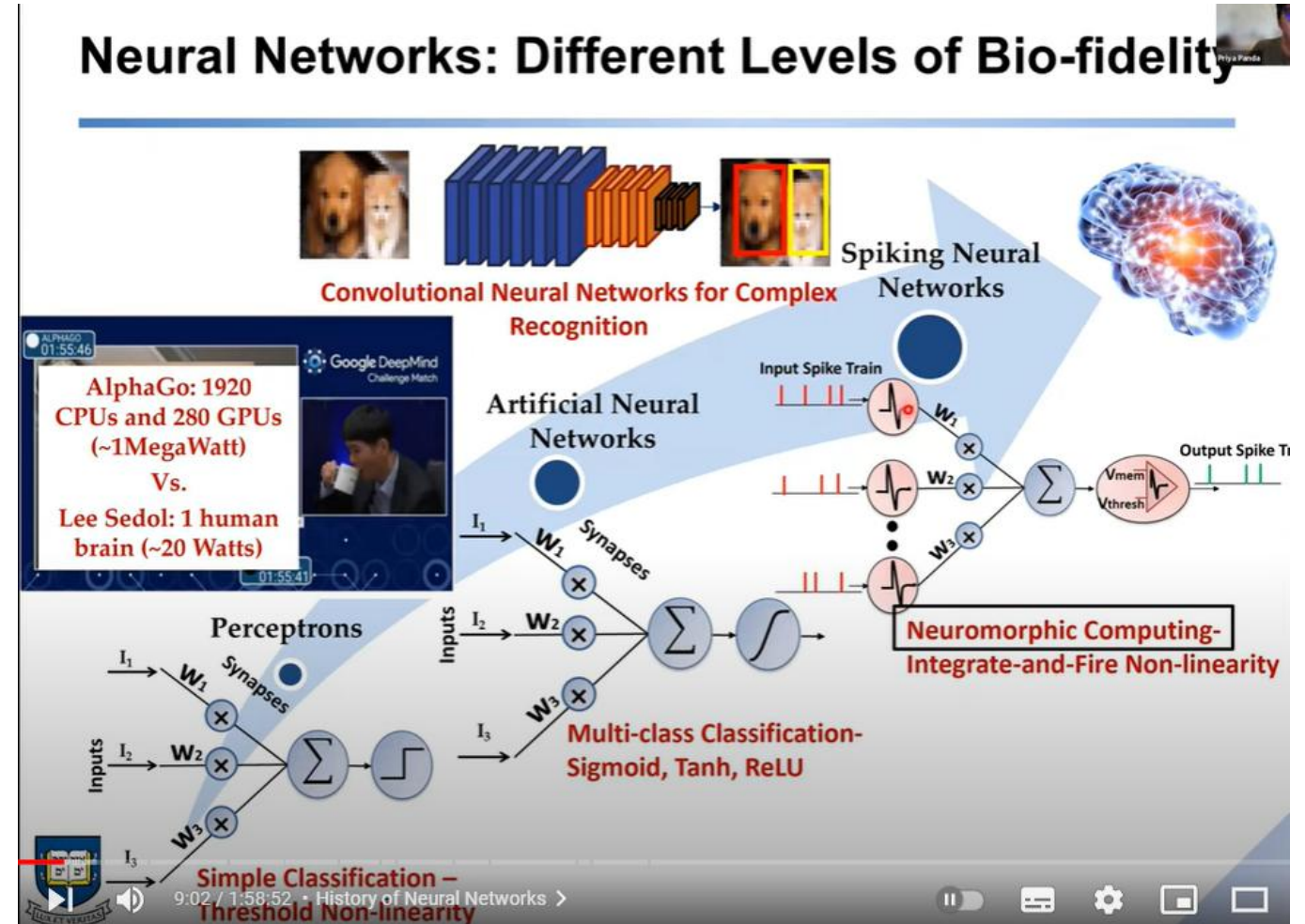
인포그래픽은 슈퍼노바 기술 개념도. SK텔레콤은 자사 전시관 전시관 5G 커넥티드 팩토리 부스에 '슈퍼노바'를 활용한 반도체 제조공정 혁신 모델을 전시한다. <SK텔레콤>

Artificial Neural Network (인공신경망)

인공 신경망

- 신경망의 종류와 발전

- 가장 단순한 형태인 퍼셉트론(Perceptron)에서 시작하여, 여러 층을 쌓은 심층 신경망(Deep Neural Network), 이미지 처리에 특화된 합성곱 신경망(CNN), 그리고 인간의 뇌를 더 유사하게 모방한 스파이킹 신경망(Spiking Neural Network) 등으로 발전.
- 신경망은 점차 복잡하고 생물학적으로 유사한 형태로 진화 중.



Perceptron (퍼셉트론)

- 퍼셉트론 (Perceptron)

- 1957년에 고안된 가장 단순한 형태의 인공 신경망으로, 다수의 입력을 받아 하나의 결과를 출력.
- 각 입력 신호($x_1, x_2, \dots$)에 고유한 가중치($w_1, w_2, \dots$)가 곱해진다. 이 신호들의 가중합 (Weighted Sum)이 특정 임계값(Threshold)을 넘으면 1을 출력하고, 넘지 못하면 0(또는 -1)을 출력하는 구조.
- 이는 뉴런이 특정 세기 이상의 자극을 받아야만 활성화되는 모습을 모방한 것임.

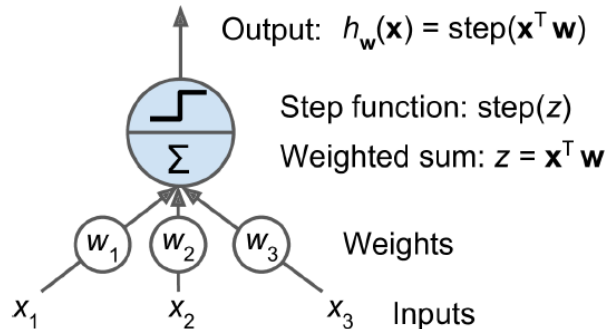
Perceptron

- **Perceptron**

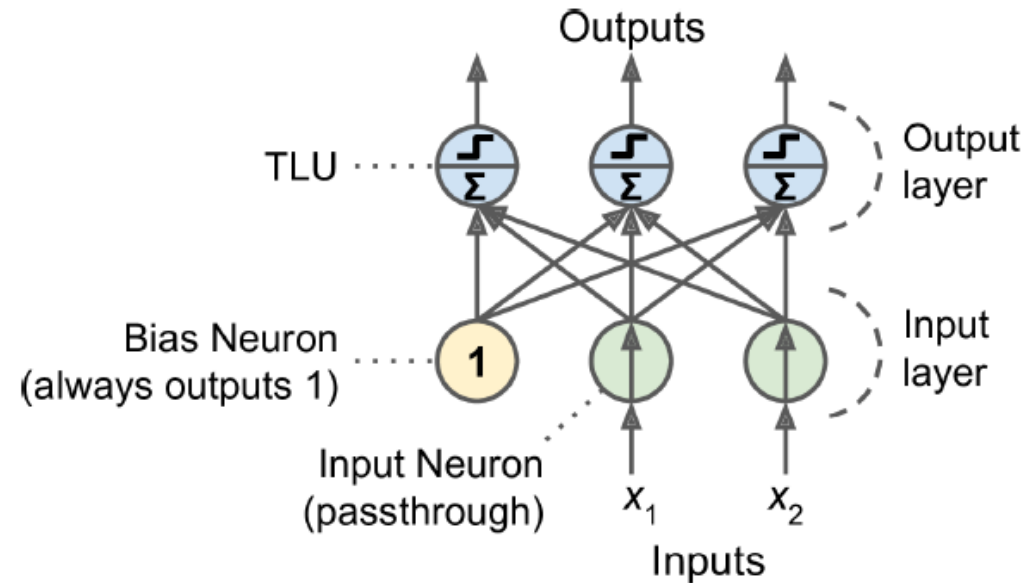
- A simplest ANN architecture based on **threshold logic unit (TLU)** or linear threshold unit (LTU) (1957)
- A single layer of TLUs (fully connected or a dense layer)

$$f(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0, \\ 0 & \text{otherwise} \end{cases}$$

Threshold logic unit

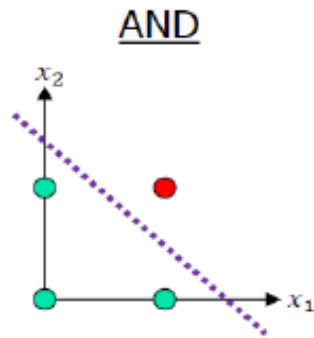
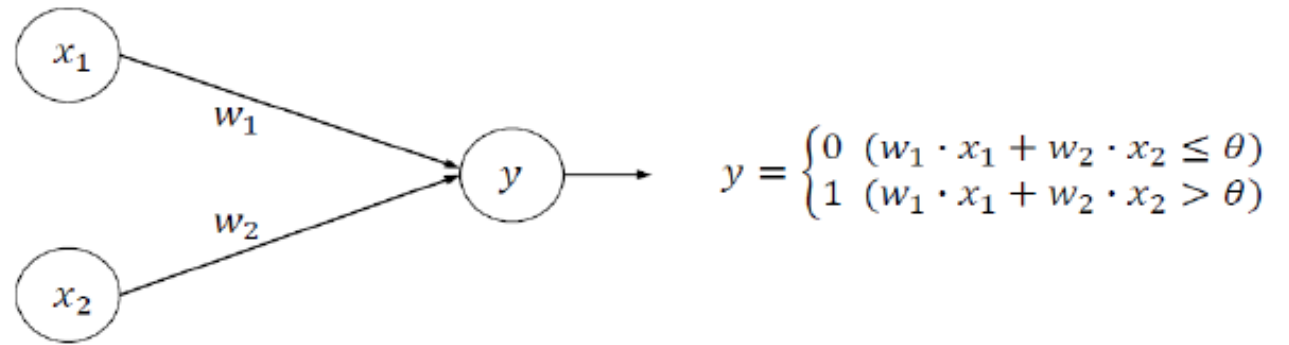


Perceptron diagram

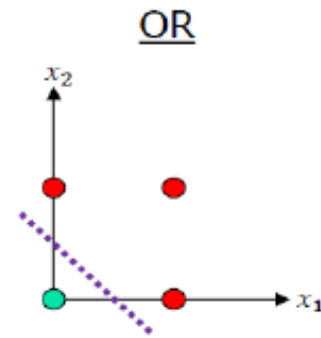


퍼셉트론의 한계: XOR 문제

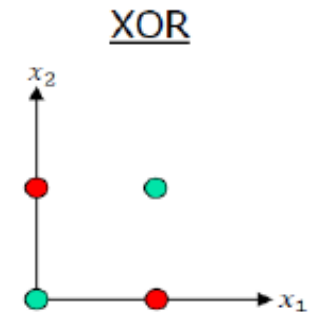
- 퍼셉트론은 선형(linear) 경계선으로 데이터를 분류하는 모델이다. 따라서, AND, OR 게이트처럼 선 하나로 두 그룹을 나눌 수 있는 문제는 해결할 수 있다.
- 하지만, XOR 문제와 같이 직선 하나로 나눌 수 없는 분류가 불가능한, 즉 비선형(non-linear) 문제에는 해결할 수 없는 명확한 한계를 가짐. (인공 신경망 연구 한동안 침체기 겪은 원인)
- But, later proved that Multilayer Perceptron (MLP) can solve the problem.



$$\begin{aligned} w_1 &= 1.0 \\ w_2 &= 1.0 \\ \theta &= 1.5 \end{aligned}$$



$$\begin{aligned} w_1 &= 1.0 \\ w_2 &= 1.0 \\ \theta &= 0.5 \end{aligned}$$

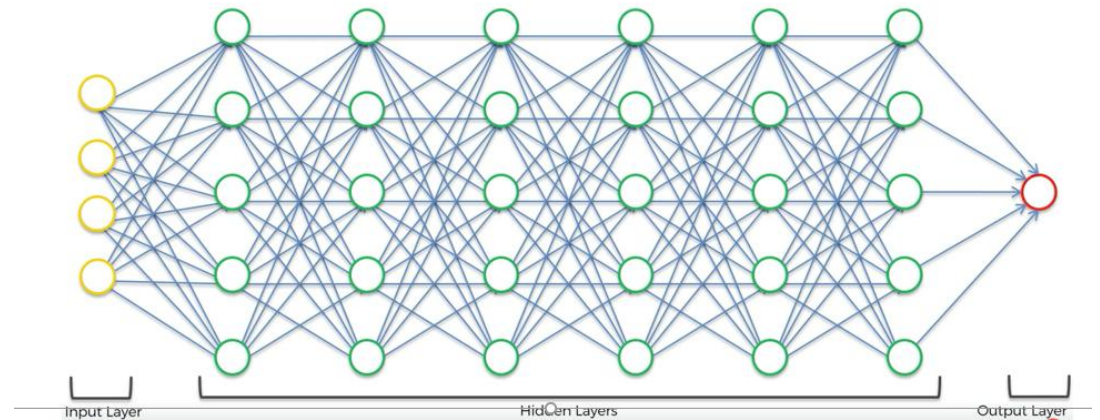
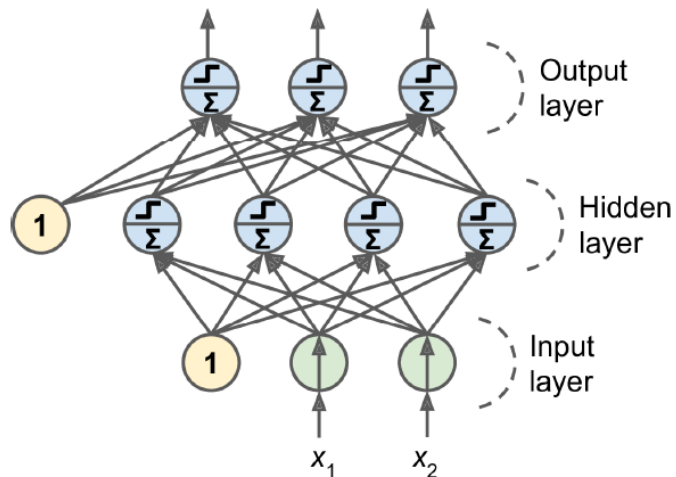


$$\begin{aligned} w_1 &=? \\ w_2 &=? \\ \theta &=? \end{aligned}$$

다층 퍼셉트론(MLP)과 XOR 문제 해결

- 다층 퍼셉트론(MLP)과 XOR 문제 해결

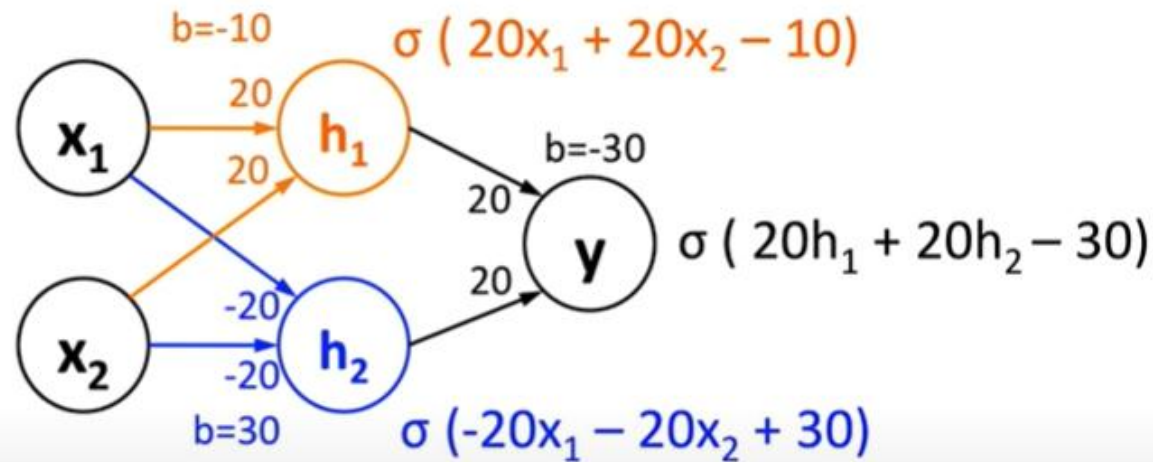
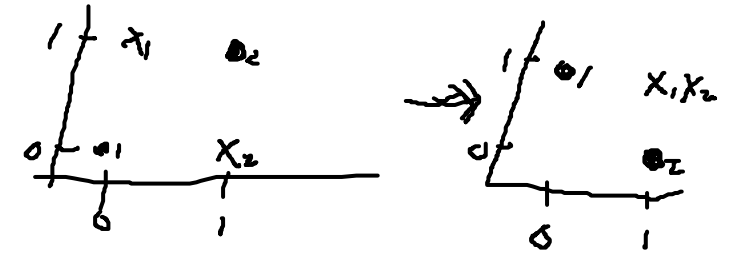
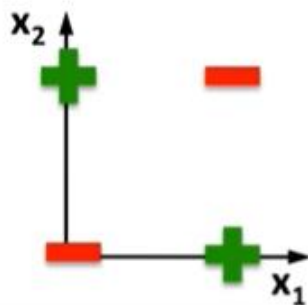
- XOR 문제는 퍼셉트론을 여러 개 쌓은 다층 퍼셉트론(Multi-Layer Perceptron, MLP)로 해결 가능.
- MLP는 입력층(Input Layer)과 출력층(Output Layer) 사이에 하나 이상의 은닉층(Hidden Layer)을 추가한 구조.
- 은닉층을 통해 입력 데이터를 비선형적인 공간으로 변환함으로써, 단층 퍼셉트론이 풀지 못했던 XOR와 같은 비선형 문제를 해결할 수 있게 됨.



MLP and XOR Problem

- Solving XOR problem using MLP

Linear classifiers cannot solve this



$\sigma(20 \cdot 0 + 20 \cdot 0 - 10) \approx 0$	$\sigma(-20 \cdot 0 - 20 \cdot 0 + 30) \approx 1$	$\sigma(20 \cdot 0 + 20 \cdot 1 - 30) \approx 0$
$\sigma(20 \cdot 1 + 20 \cdot 1 - 10) \approx 1$	$\sigma(-20 \cdot 1 - 20 \cdot 1 + 30) \approx 0$	$\sigma(20 \cdot 1 + 20 \cdot 0 - 30) \approx 0$
$\sigma(20 \cdot 0 + 20 \cdot 1 - 10) \approx 1$	$\sigma(-20 \cdot 0 - 20 \cdot 1 + 30) \approx 1$	$\sigma(20 \cdot 1 + 20 \cdot 1 - 30) \approx 1$
$\sigma(20 \cdot 1 + 20 \cdot 0 - 10) \approx 1$	$\sigma(-20 \cdot 1 - 20 \cdot 0 + 30) \approx 1$	$\sigma(20 \cdot 1 + 20 \cdot 1 - 30) \approx 1$

[Ref: <https://www.youtube.com/watch?v=kNPGXgzxoHw>]

Activation Functions (활성화 함수)

- **활성화 함수 (Activation Function)**

- 뉴런의 가중합 결과를 출력 신호로 변환하는 함수
- 초기 퍼셉트론은 계단 함수(Step Function)를 사용했는데, 이는 미분이 불가능하여 가중치를 점진적으로 학습시키기 어려웠음.
- 이 문제를 해결하기 위해, 미분 가능한 다양한 비선형 함수들이 활성화 함수로 제안됨.

- **대표적인 활성화 함수**

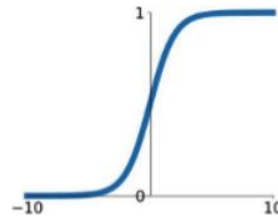
- 시그모이드(Sigmoid): 출력을 0과 1 사이로 압축한다. 주로 이진 분류 문제의 출력층에서 확률을 나타낼 때 사용.
- 하이퍼볼릭 탄젠트(Tanh): 출력을 -1과 1 사이로 압축한다. 시그모이드보다 중심값이 0이라 더 효율적인 학습이 가능할 때가 많다.
- ReLU (Rectified Linear Unit): 입력이 0보다 크면 그대로 출력하고, 0보다 작으면 0을 출력한다. 계산이 간단하고 학습 속도가 빨라 현재 은닉층에서 가장 널리 사용되는 활성화 함수.

Activation Functions

- **Changed Activation function**
 - step function is flat, so there is no gradient.
 - There are several activation functions.

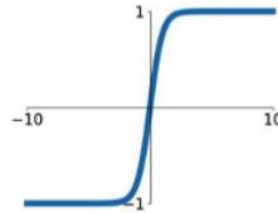
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



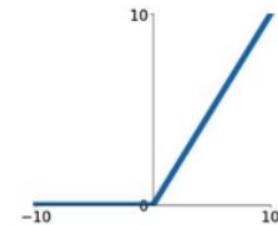
tanh

$$\tanh(x)$$



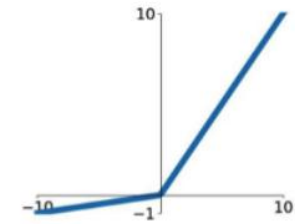
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

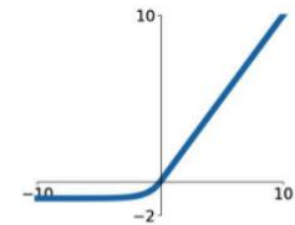


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



활성화 함수와 Gradient(도함수)

- **활성화 함수 (Activation Function)**

- 신경망이 가중치를 학습하는 과정(역전파)에서는 활성화 함수의 미분값(그래디언트) 사용.
- 계단 함수는 대부분의 구간에서 미분값이 0이므로 학습이 진행되지 않는다.
- 시그모이드(Sigmoid)와 Tanh 함수 역시 입력값의 절댓값이 커지면 그래프의 기울기가 0에 가까워지는 **그래디언트 소실 문제(Vanishing Gradient Problem)**가 발생한다. 이 때문에 은닉층이 깊어질수록 앞쪽 레이어까지 그래디언트가 잘 전달되지 않아 학습 효율이 떨어질 수 있다.
- 반면, ReLU 함수는 입력값이 양수일 때 미분값이 항상 1이므로, 그래디언트 소실 문제에서 비교적 자유로워 깊은 신경망의 은닉층에서 훨씬 효율적인 학습이 가능하다.
 - Relu 의 음수 영역: 뉴런의 활성화를 억제하는 반대 방향의 자극으로 이해할 수 있음. (실제 뇌의 뉴런은 흥분성(Excitatory) 신호와 억제성(Inhibitory) 신호 두 종류의 신호를 받는다고 함. 가중합($XW+b$) 이 양수는 흥분성, 음수는 억제성으로 해석)
 - 실제 뉴런도 마이너스 강도로 신호를 보내지 않고, 뉴런은 **발화하거나(fire) 발화하지 않거나(not fire)** 둘 중 하나의 행동을 한다. 여기서 '발화하지 않는 상태'가 바로 억제성 신호가 흥분성 신호보다 강했거나, 흥분성 신호가 활성화 임계값을 넘기에 충분히 강하지 않았다는 '결과'를 의미한다.

Activation Functions and Derivatives

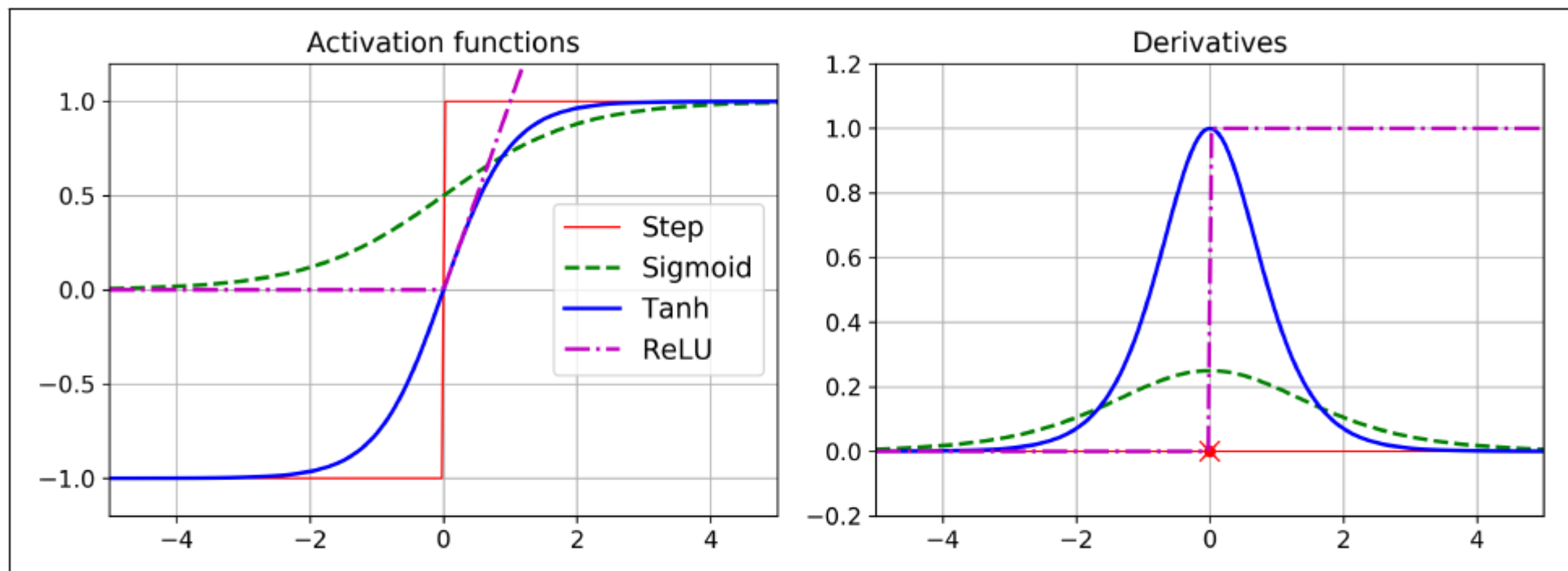


Figure 10-8. Activation functions and their derivatives

MLP 응용: Regression

- **MLP 응용 1 - 회귀 (Regression)**

- 회귀는 주가, 온도, 점수 등 **연속적인 값**을 예측하는 문제
- 입력층: 예측에 사용할 특징(feature)의 수만큼 뉴런을 둔다.
- 은닉층: 보통 1~5개의 층을 사용하며, 각 층의 뉴런은 10~100개 사이로 설정한다. 활성화 함수로는 주로 ReLU를 사용한다.
- 출력층: 예측하려는 값의 차원 수만큼 뉴런을 둔다 (보통 1개). 예측값이 특정 범위에 있을 필요가 없다면 활성화 함수를 사용하지 않는다.
- 손실 함수: 예측값과 실제값의 차이를 측정하기 위해 주로 평균 제곱 오차(MSE)를 사용한다.

MLP for regression

Table 10-1. Typical Regression MLP Architecture

Hyperparameter	Typical Value
# input neurons	One per input feature (e.g., $28 \times 28 = 784$ for MNIST)
# hidden layers	Depends on the problem. Typically 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU, see Chapter 11)
Output activation	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

MLP 응용: Classification

- **MLP 응용 2 - 분류 (Classification)**

- 분류는 스팸 메일 여부, 숫자 종류 등 주어진 데이터를 정해진 카테고리로 나누는 문제.
- 입력층과 은닉층의 구성은 회귀 문제와 유사하다.
- 출력층의 활성화 함수와 손실 함수가 문제 유형에 따라 달라진다.
 - 이진 분류(Binary Classification): 두 개 중 하나를 선택하는 문제. 출력 뉴런 1개와 시그모이드(로지스틱) 함수를 사용.
 - 다중 클래스 분류(Multi-class Classification): 여러 개 중 단 하나를 선택하는 문제. 각 클래스마다 뉴런을 하나씩 두고, 출력 확률의 총합이 1이 되도록 하는 소프트맥스(Softmax) 함수를 사용.
- 손실 함수: 모든 분류 문제에서 일반적으로 **교차 엔트로피(Cross-Entropy)**를 사용한다.

Multi-Class Classification and Softmax

- **Multi-Class Classification and Softmax**

- 3개 이상의 클래스를 분류할 때, 출력층은 각 클래스에 대한 ****점수(score)****를 계산한다.
- **소프트맥스(Softmax)** 함수는 이 점수들을 입력받아, 모든 클래스에 대한 확률 값으로 변환해준다.
- 변환된 값들은 0과 1 사이이며, 모든 값을 더하면 항상 1이 된다.
- 모델은 이 확률 값들 중 가장 높은 값을 가진 클래스를 최종 예측 결과로 선택한다.

MLP for classification

MLP (including ReLU and softmax) for classification

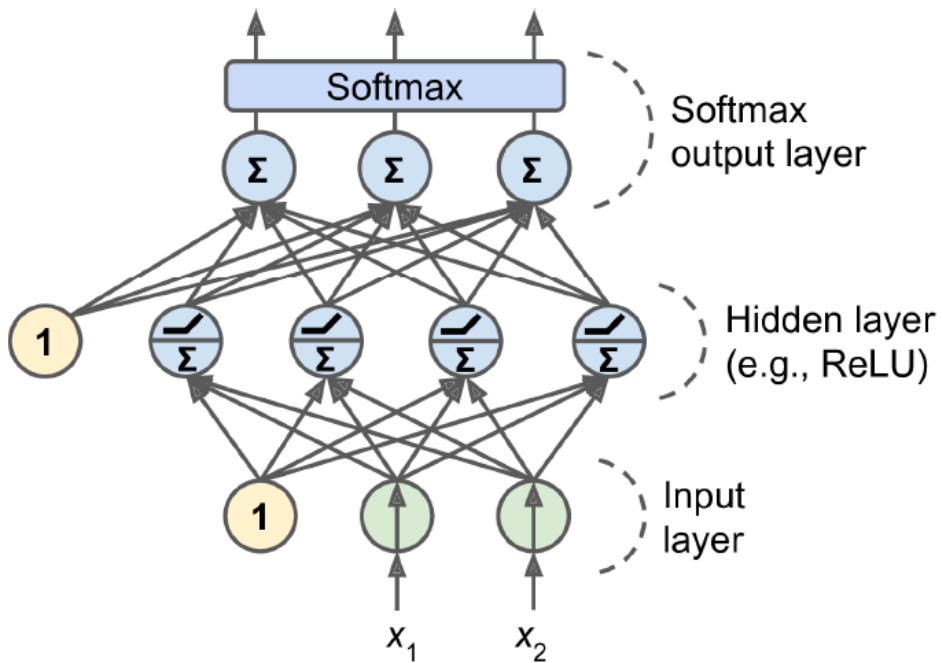


Table 10-2. Typical Classification MLP Architecture

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

multi-class: single label multi-class, (e.g. [0] or [1] or [2])
multi-label: may have more than one class label
(e.g. [1,0,1], [1,1,0])

Deep Learning Training

Training(학습) and Loss Function(손실함수)

- **딥러닝 모델의 학습:**

- 초기 신경망 모델은 아직 훈련되지 않은 초기 상태이다. 학습(Training)은 이 모델이 정답을 잘 맞힐 수 있도록 내부의 **가중치(Weight)**와 **편향(Bias)**을 점진적으로 업데이트하는 과정이다. 이 학습 과정은 세 가지 핵심 요소로 이루어진다.
 - **손실 함수 (Loss Function):** 모델이 얼마나 틀렸는지 '오차'를 측정
 - **역전파 (Backpropagation):** 이 오차에 각 가중치가 얼마나 기여했는지('책임')를 계산
 - **최적화 알고리즘 (Optimizer):** 계산된 책임을 바탕으로 가중치를 업데이트

- **손실 함수 (Loss Function)**

- 손실 함수는 모델의 예측 값과 실제 정답이 얼마나 다른지를 측정하는 '오답 노트'와 같다. 손실 함수의 결과값(Loss)이 크면 모델이 많이 틀렸다는 의미이고, 작으면 정답에 가깝게 예측했다는 의미이다.
- 딥러닝 학습의 목표는 이 손실 함수의 값을 가능한 0에 가깝게 만드는 것이다. 이를 위해 모델은 손실 값을 줄이는 방향으로 자신의 가중치를 계속해서 수정해 나간다.

분류 문제의 손실 함수: 교차 엔트로피 (Cross-Entropy)

- **분류 문제의 손실 함수: 교차 엔트로피 (Cross-Entropy):**
 - 분류 문제에서는 주로 교차 엔트로피(Cross-Entropy) 손실 함수를 사용한다.
 - 소프트맥스(Softmax) 함수가 모델의 예측을 '확률 분포'로 만들어준다. (예: [고양이일 확률: 70%, 개일 확률: 20%, 토끼일 확률: 10%])
 - 정답 또한 확률 분포로 표현할 수 있다. (예: [고양이일 확률: 100%, 개일 확률: 0%, 토끼일 확률: 0%])
 - 교차 엔트로피는 모델이 예측한 확률 분포와 정답의 확률 분포가 얼마나 다른지를 측정하는 지표이다. 두 분포가 비슷할수록 값은 작아지고, 다를수록 값은 커진다.

Softmax and Cross-Entropy

- **소프트맥스 + 교차 엔트로피 조합:**

- 다중 클래스 분류 모델에서 소프트맥스 활성화 함수와 교차 엔트로피 손실 함수는 거의 항상 함께 사용되는 표준적인 조합.
 - 입력 데이터가 신경망을 통과하여 각 클래스에 대한 ****점수(Score)****를 출력한다.
 - 출력층의 소프트맥스 함수가 이 점수를 확률로 변환한다.
 - 교차 엔트로피 함수가 이 예측 확률과 실제 정답을 비교하여 최종 손실(Loss) 값을 계산한다.

Backpropagation and Optimizer

- **역전파 알고리즘 (Backpropagation):**

- 역전파: 모델이 얼마나 틀렸는지를 손실함수로 표현하고, 이 오차의 원인을 거꾸로 추적하여 각 가중치가 오차에 얼마나 기여했는지, 즉 책임을 계산하는 과정.
- 수학적으로는 미분의 연쇄 법칙(Chain Rule)을 사용하여, 손실 함수에 대한 모든 가중치의 그래디언트(Gradient)를 효율적으로 계산하는 알고리즘.
- 계산된 그래디언트 값은 '손실을 줄려면 이 가중치를 어느 방향으로 얼마나 조절해야 하는지'에 대한 정보를 담고 있다.

- **최적화 알고리즘 (Optimizer):**

- 역전파를 통해 계산된 그래디언트 정보를 사용하여, 실제로 모델의 가중치를 업데이트하는 역할
- 즉, 손실 함수라는 산의 가장 낮은 지점(최솟값)을 찾아가는 등산객의 발걸음과 같다.

Gradient Descent, GD (경사 하강법)

- **경사 하강법 (Gradient Descent, GD):**
 - 가장 기본적인 최적화 알고리즘.
 - 현재 위치에서 손실 함수 값(산의 높이)이 가장 가파르게 감소하는 방향, 즉 **그래디언트의 반대 방향**으로 한 걸음씩 이동하는 방식.
 - 이 과정을 반복하면 결국 손실이 가장 낮은 지점(Global Minimum 또는 Local Minimum)에 도달하게 된다.
 - **업데이트 공식:** $W_{new} = W_{old} - \eta \times \nabla J(W)$
 - (η : 학습률, $\nabla J(W)$: 손실 함수에 대한 가중치의 그래디언트)

Learning Rate, η (학습률)

- **학습률 (Learning Rate, η):**
 - 경사 하강법에서 한 번에 얼마나 큰 걸음으로 이동할지를 결정하는 매우 중요한 하이퍼파라미터
 - 학습률이 너무 크면 (high): 최적점을 그냥 지나쳐 버려 값이 발산(diverge)할 수 있다.
 - 학습률이 너무 작으면 (low): 최적점을 그냥 지나쳐 버려 값이 발산(diverge)할 수 있다.
 - 따라서, 적절한 학습률을 찾는 것이 중요하다.

Stochastic GD and Mini-Batch

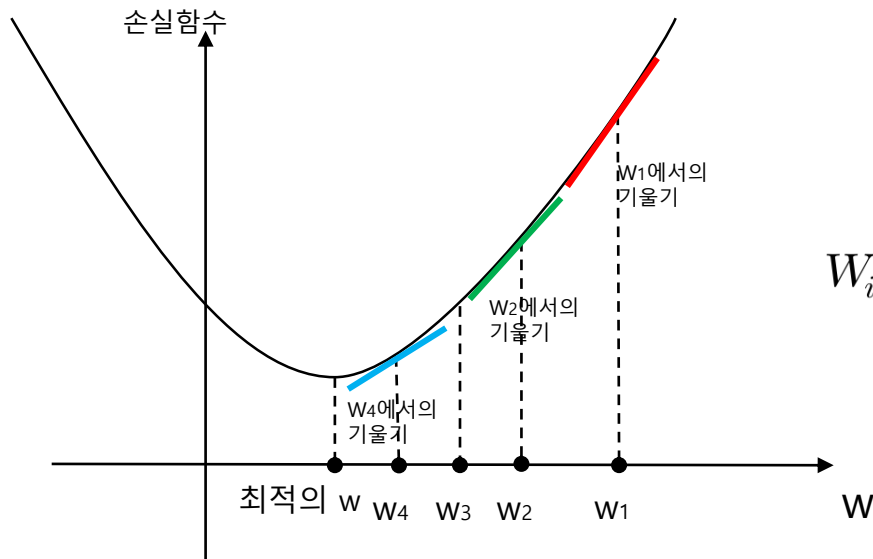
- **경사 하강법의 한계:**
 - 기본 경사 하강법(배치 경사 하강법)은 한 걸음을 내딛기 위해 **전체 학습 데이터**를 모두 계산에 사용해야 한다. 데이터가 수백만 개 이상인 딥러닝에서는 이 방식이 비효율적이고 너무 느리다는 단점이 있다.
- **확률적 경사 하강법 (SGD)과 미니배치:**
 - 확률적 경사 하강법 (Stochastic Gradient Descent, SGD): 전체 데이터가 아닌, 랜덤하게 뽑은 데이터 1개만 보고 가중치를 업데이트한다. 속도는 매우 빠르지만, 업데이트 방향이 불안정하여 손실 값이 심하게 요동칠 수 있다.
 - 미니배치 경사 하강법 (Mini-batch GD): 전체 데이터와 1개 데이터의 절충안으로, 작은 묶음(mini-batch, 보통 32~256개)의 데이터를 보고 가중치를 업데이트한다. 안정적이면서도 속도가 빨라, 현재 딥러닝에서 가장 표준적으로 사용되는 방식이다.

Optimization Algorithms

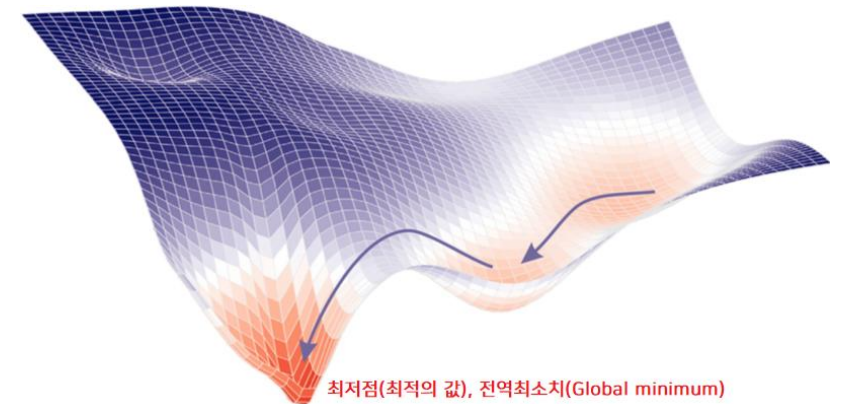
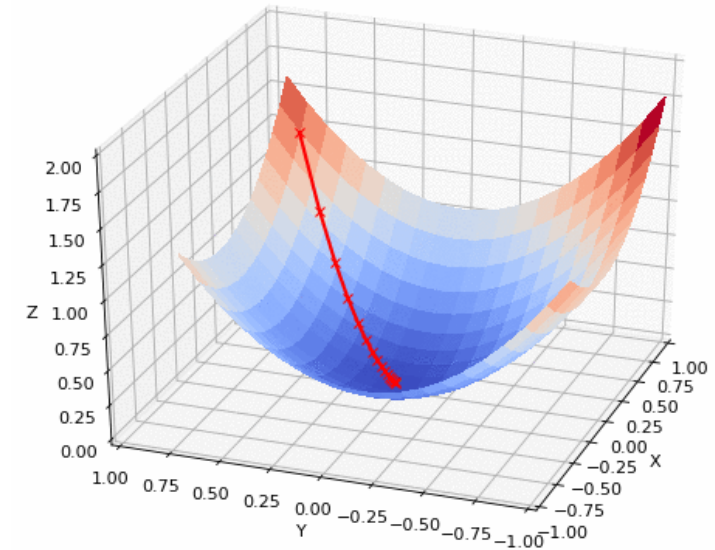
Optimization algorithms

- **Gradient Descent** (경사하강법)

- General optimization algorithm
- take repeated steps in the opposite direction of the **gradient** (or approximate **gradient**) of the function at the current point



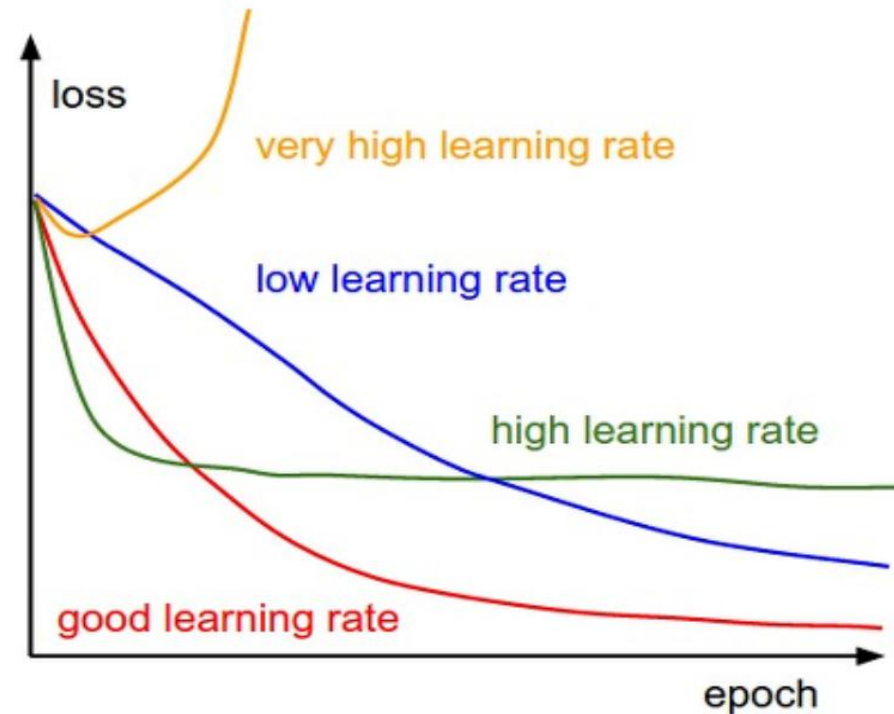
$$W_i = W_{i-1} - \eta \text{Grad}(i)$$



Optimization algorithms

- **Learning rate: η (eta)**

- low: takes time to converge, and may get stuck in an undesirable local minimum
- high: may jump over minima
- too high: may diverge
- Need adaptive adjustment



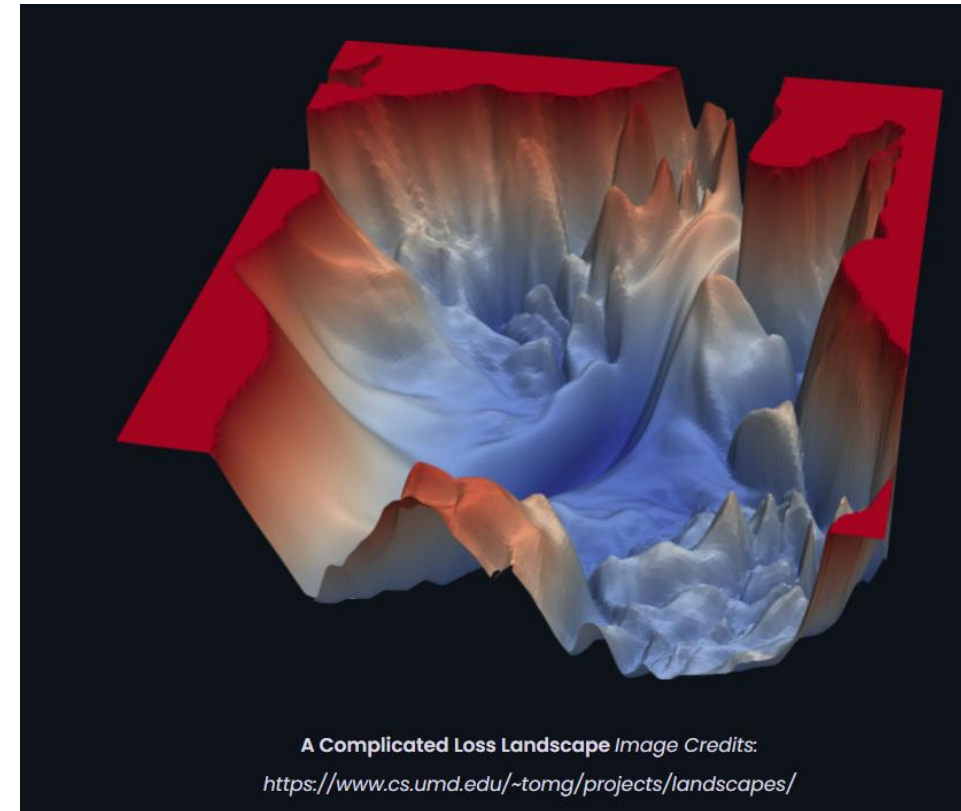
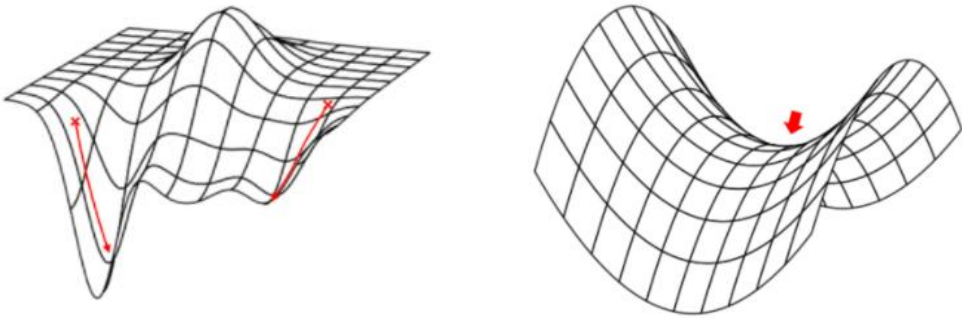
Optimization algorithms

- **Gradient descent**

- computes the gradient of the cost function w.r.t. to the parameters θ for the **entire** training dataset for each update (too much computation)

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$$

- Local minimum
- Saddle Point

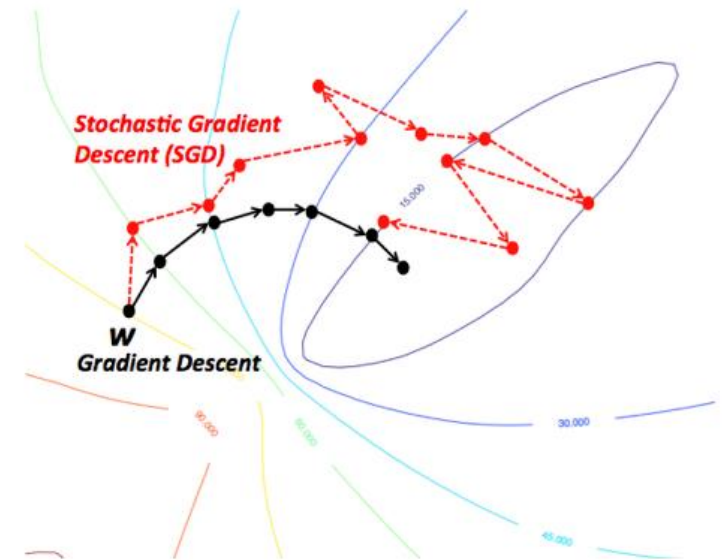


Optimization algorithms

- **Stochastic gradient descent (SGD)**

- performs a parameter update for **each** training example $x^{(i)}$ and label $y^{(i)}$
- usually much faster and can also be used to learn online
- It performs frequent updates with a high variance that cause the objective function to fluctuate heavily.
- higher probability not to fall in local minima

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$



(ex) sklearn 의 SGDRegressor(), SGDClassifier()

- 100개 샘플이 있으면 1 epoch 안에 가중치 업데이트가 100번 일어난다.

Optimization algorithms

- **Mini-batch gradient descent**

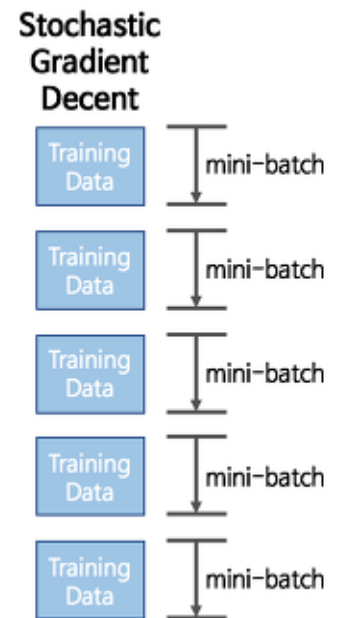
- performs an update for every **mini-batch** of n training examples
- common mini-batch sizes range between 50 and 256, but can vary for different applications

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$$

(ex) [Keras/PyTorch](#) "SGD optimizer"

- **Challenges**

- GD does **not guarantee good convergence**, but offers few challenges.
- How to find **optimal learning rates**? -> decaying (step size)
- How often should the parameters be **updated**? -> batch size
- How to **escape from local minima**? -> momentum (step direction)

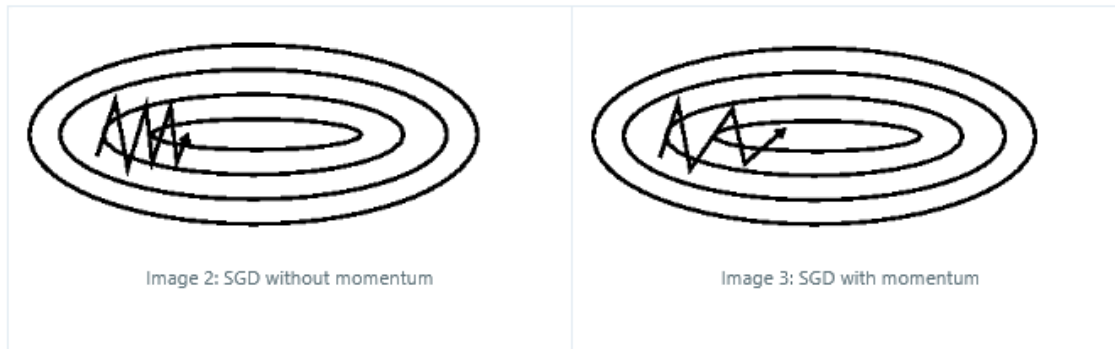


Problems in Optimization

- 미니배치 경사 하강법의 문제점:
 - 손실 함수의 표면이 길고 좁은 골짜기 형태일 때, 골짜기 양옆으로 진동하며 비효율적으로 탐색할 수 있다.
 - 최적점이 아닌 움푹 팬 지점(local minimum)이나 평평한 안장점(saddle point)에 빠져 학습이 멈출 수 있다.
- 이 문제들을 해결하기 위해, 단순히 기울기 반대 방향으로만 움직이는 것을 넘어 더 현명하게 움직이는 방법들이 고안되었다. 핵심 아이디어는 걸음의 방향과 걸음의 크기를 동적으로 조절하는 것이다.

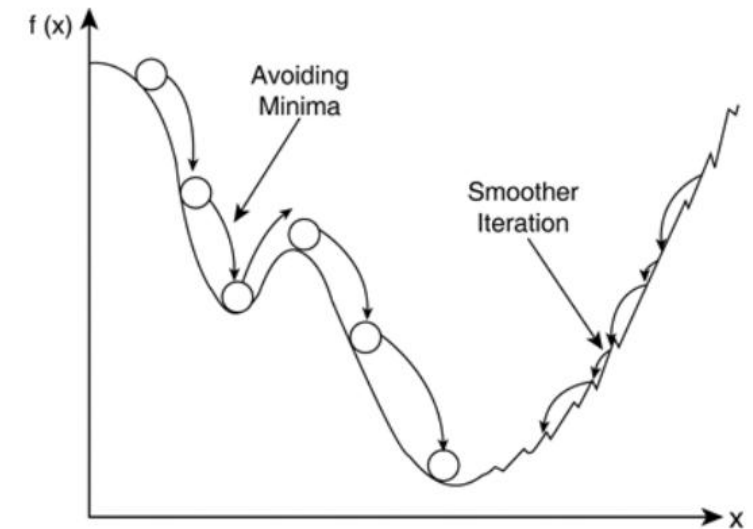
Optimization algorithms

- **Momentum (batch GD with Momentum)**
 - movement(update) vector (v) introduced
 - Also use the results from previous batch (momentum term m is close to 0.9)
 - Converge faster, reduced vibration, and helps to escape from local minimums.



$$V_t = m \times V_{t-1} - \eta \nabla_{\omega} J(\omega_t)$$

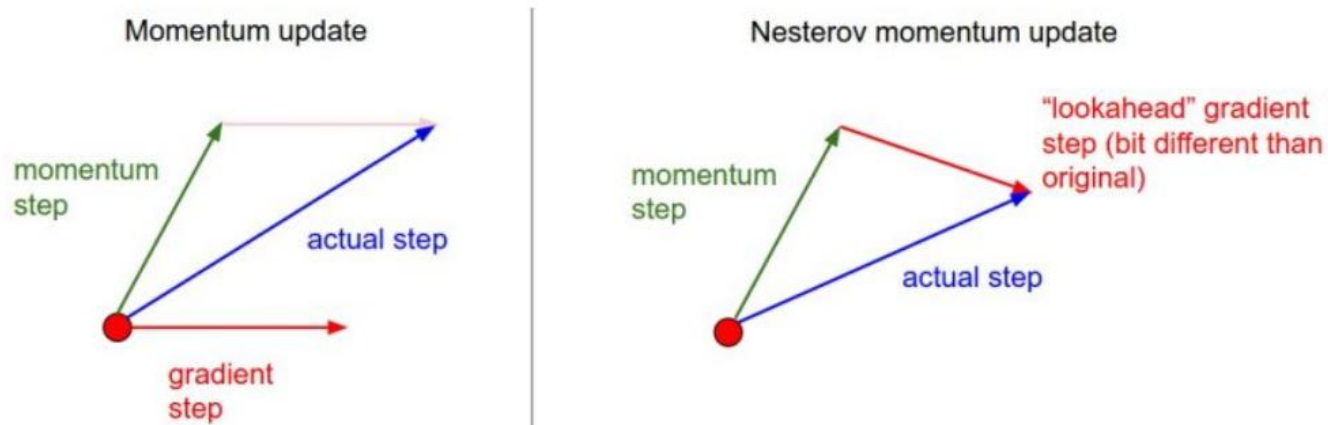
$$\omega_{t+1} = \omega_t + V_t$$



Avoiding Local Minima. Picture from <http://www.yalDEX.com>.

Optimization algorithms

- **Improved Momentum (Nesterov accelerated gradient (NAG))**
 - Momentum 방법과 동일하지만 Gradient 계산이 약간 다름
 - Momentum 에서는 현재 위치의 기울기와 모멘텀을 따로 계산하고 나중에 더하지만, 여기서는 먼저 모멘텀을 계산 후 기울기 를 계산



Difference between Momentum and NAG. Picture from CS231.

$$V_t = m \times V_{t-1} - \eta \nabla_{\omega} J(\omega_t - m \times V_{t-1})$$

$$\omega_{t+1} = \omega_t + V_t$$

적응형 학습률 (Adaptive Learning Rates)

- Separate adaptive learning rates: 학습률을 weight 에 따라 다르게 함.
- **Adagrad** (Adaptive gradient)
 - 파라미터 별 update (different learning rates for ω_i 's)
 - 과거에 많이 변경되지 않은 매개 변수에 더 큰 learning rate 적용 -> step-size 감소
 - 작동 방식: 각 파라미터마다 '지금까지 변화해온 양(그라디언트 제곱의 합)'을 기록한다. 그리고 이 값이 클수록 학습률을 작게, 작을수록 학습률을 크게 조절한다.
 - 아래 식에서 squaring 과 dot(.) 연산은 element-wise 연산

$$G_t = G_{t-1} + (\nabla_{\omega} J(\omega_t))^2 = \sum_{i=1}^k (\nabla_{\omega_i} J(\omega_i))^2$$
$$\omega_{t+1} = \omega_t - \frac{\eta}{\sqrt{G_t} + \epsilon} \cdot \nabla_{\omega} J(\omega_t)$$

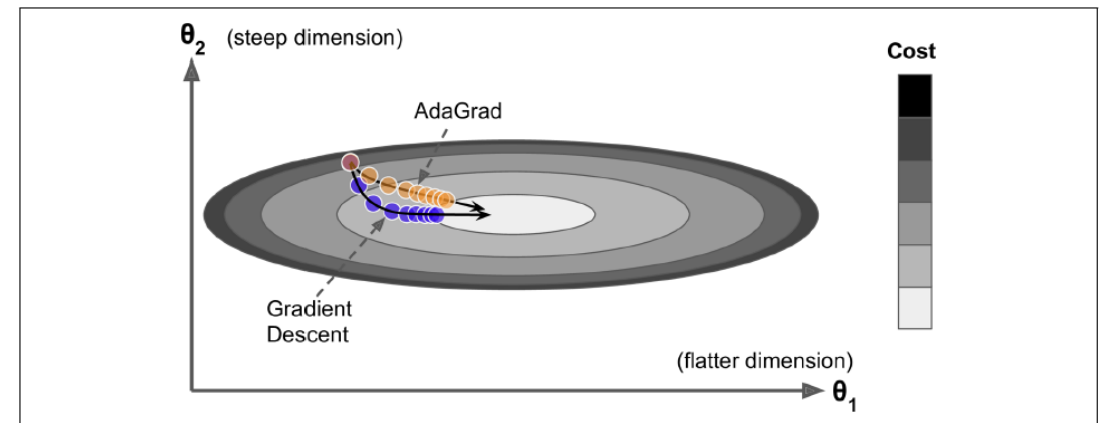


Figure 11-7. AdaGrad versus Gradient Descent

ref: Hands-on-Machine Learning (2nd)

Optimization algorithms

- **RMSProp** (Root Mean Square Propagation)

- Adagrad 알고리즘은 너무 급격히 감소하여 global optimum 에 도달하지 못하는 경우 발생.
- 처음부터 모든 gradient G_t 를 합산하는 대신 **지수 이동 평균** (exponential moving average) 사용하여 최근 것 사용 (more weights on the recent gradient): typical decay rate $\gamma = 0.9 \sim 0.999$
- Always performs much better than Adagrad, and most preferred until Adam came around.

$$G_t = \gamma G_{t-1} + (1 - \gamma)(\nabla_{\omega} J(\omega_t))^2$$

$$\omega_{t+1} = \omega_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla_{\omega} J(\omega_t)$$

```
optimizer = keras.optimizers.RMSprop(lr=0.001, rho=0.9)
```

gamma



Optimization algorithms

- **Adam** (Adaptive Moment Estimation)

- RMSProp + Momentum
- Momentum 과 유사하게 기울기의 지수 평균 반영 (과거 기울기 (past gradient) 의 지수적으로 감소하는 평균을 유지)
- RMSProp 과 유사하게 기울기 제곱 값의 지수 평균 반영 (각 매개 변수에 대한 적응형 학습률 (adaptive learning rate))
- Recommended values: $\epsilon=10^{-8}$, $\beta_1=0.9$, $\beta_2=0.999$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\omega} J(\omega_t) \quad \text{first moment (관성)}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\omega} J(\omega_t))^2 \quad \text{second moment(학습률)}$$

$$\omega_{t+1} = \omega_t - m_t \frac{\eta}{\sqrt{v_t + \epsilon}}$$

$$\begin{aligned} M(t) &= \beta_1 M(t-1) + (1 - \beta_1) \frac{\partial}{\partial w(t)} \text{Cost}(w(t)) \\ V(t) &= \beta_2 V(t-1) + (1 - \beta_2) \left(\frac{\partial}{\partial w(i)} \text{Cost}(w(i)) \right)^2 \\ \hat{M}(t) &= \frac{M(t)}{1 - \beta_1^t} \quad \hat{V}(t) = \frac{V(t)}{1 - \beta_2^t} \\ W(t+1) &= W(t) - \alpha * \frac{\hat{M}(t)}{\sqrt{\hat{V}(t) + \epsilon}} \end{aligned}$$

- bias correction terms (exponential decay)
- 일반적으로 m, v 가 0 으로 초기화되어 있기 때문에 훈련 초기에 크게 bias 되는데 이를 교정하기 위해 bias correction term 이 필요함.
- 훈련 iteration 이 진행 되면서 bias correction term 은 1 로 접근하게 됨.

Optimization algorithms

- **Adam** (Adaptive Moment Estimation)

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

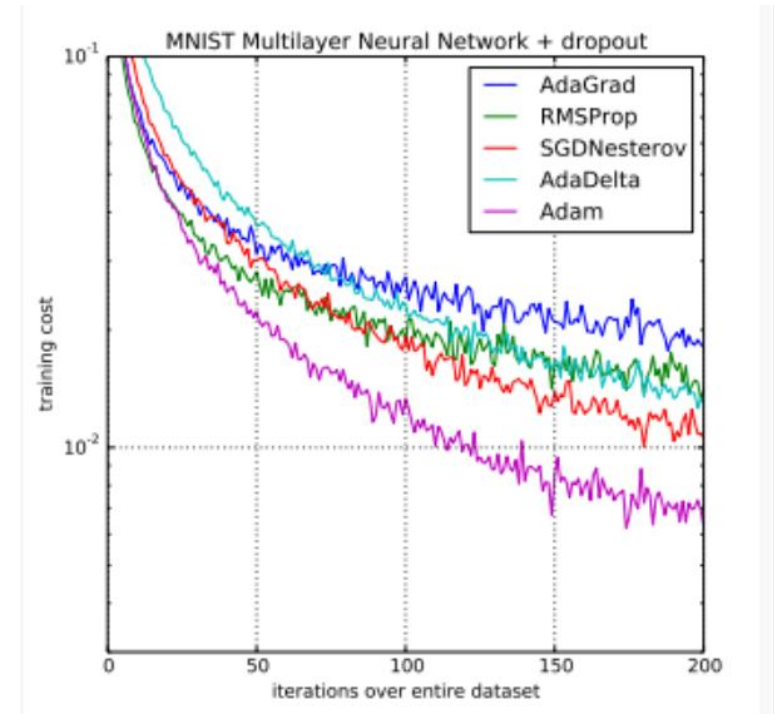
$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

return θ_t (Resulting parameters)

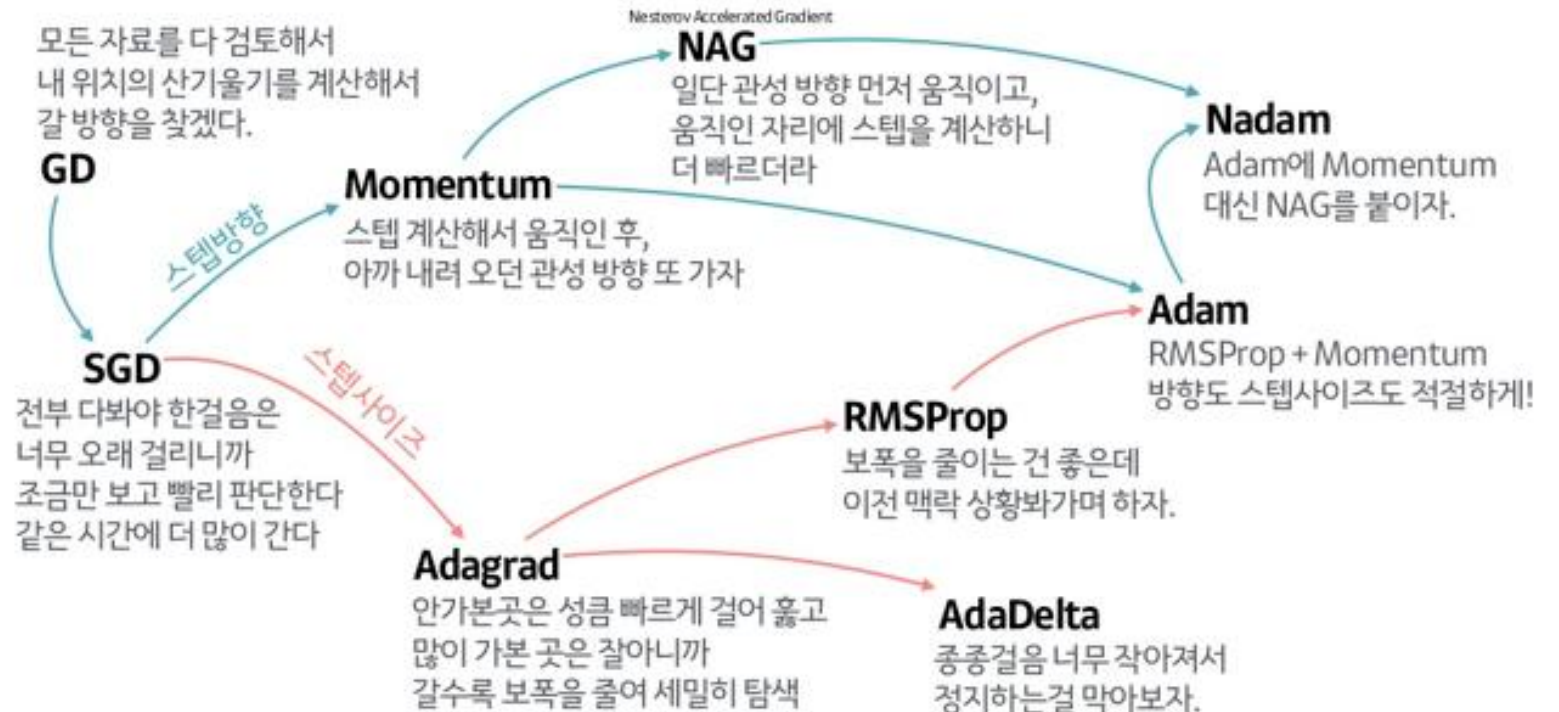


Comparison of Adam to Other Optimization Algorithms Training a Multilayer Perceptron

Taken from Adam: A Method for Stochastic Optimization, 2015.

Optimization algorithms

- Summary



- More:

- Adamax
- Nadam
- AMSGrad
- more ...

- Good animation

- <https://towardsdatascience.com/complete-guide-to-adam-optimization-1e5f29532c3d>

Tensorflow/Keras and Pytorch

Tensorflow/Keras and Pytorch

- 가장 근본적인 철학 차이 - "그래프를 언제, 어떻게 만드는가"

	TensorFlow/Keras	PyTorch
원래 철학 (TF1)	Define-and-run (그래프를 먼저 통째로 정의하고, 나중에 데이터를 흘려 보내 실행시킴) - session.run()	Define-by-run (코드를 한 줄씩 실행하면서 그때그때 그래프가 만들어짐)
현재 (TF2/Keras 3)	기본은 eager 모드 (한 줄씩 즉시 실행시킴) 필요시 @tf.function으로 그래프 컴파일	항상 eager (torch.compile로 선택적 컴파일)

Tensorflow/Keras and Pytorch

- 미분(gradient)을 계산하는 방식 — 가장 실질적인 차이

TensorFlow/Keras	PyTorch
"테이프"가 별도로 구간을 녹음함.	텐서 자체가 "기억"을 갖고 있음.
<pre>x = tf.Variable(3.0) with tf.GradientTape() as tape: y = x ** 2 grad = tape.gradient(y, x) # 6.0</pre>	<pre>x = torch.tensor(3.0, requires_grad=True) y = x ** 2 y.backward() # y로부터 거꾸로 타고 올라가며 미분 print(x.grad) # 6.0</pre>
- 텐서 자신이 아니라 GradientTape이라는 녹음기가 with 블록 안의 연산들을 지켜보고 있다가, 나중에 tape.gradient(y, x)로 미분을 꺼내 준다. - 어디서부터 어디까지 미분을 기록할지"를 with 블록으로 명시적으로 구간을 지정 (TF는 tape이 with 블록을 벗어나면 자동으로 그 역할이 끝난다.)	- requires_grad=True인 텐서는 자기가 어떤 연산을 거쳐 만들어졌는지 계산 그래프를 스스로 기록하고 있다가, .backward()를 부르면 그 기록을 따라 거꾸로 미분한다. - 구간 지정 없이, 그냥 연산을 하면 자동으로 기록되고 .backward() 시점에 전체를 거슬러 올라간다. (그래서 PyTorch는 매 스텝 optimizer.zero_grad()로 이전 gradient를 직접 지워줘야 한다.)

Tensorflow/Keras and Pytorch

- 모델을 정의하는 방식 – call() vs. forward()

TensorFlow/Keras	PyTorch
<pre>class MyModel(keras.Model): def __init__(self): super().__init__() self.dense = keras.layers.Dense(10) def call(self, x): # 순전파 로직 return self.dense(x)</pre>	<pre>class MyModel(torch.nn.Module): def __init__(self): super().__init__() self.linear = torch.nn.Linear(20, 10) def forward(self, x): # 순전파 로직 return self.linear(x)</pre>
- 이름만 다르지 역할은 완전히 동일하다. 둘 다 "실제 계산 로직"만 여기 적고, model(x)처럼 호출했을 때 앞뒤로 부수적인 처리(build, hook 등)를 해주는 건 각 프레임워크의 __call__()이 대신한다.	

Tensorflow/Keras and Pytorch

- 학습 루프 — "자동으로 돌려줄까, 내가 다 짤까"

TensorFlow/Keras	PyTorch
- .fit() 라는 고수준의 API 제공 - 배치 반복, gradient 계산, 가중치 업데이트, 진행률 표시까지 다 대신 해주고, 우리는 필요할 때만 train_step()을 오버라이드해서 그 안쪽만 손대면 된다. - 빠르게 결과를 얻기 좋음(표준 케이스는 몇 줄로 끝남)	- 아래처럼 학습 루프를 처음부터 직접 쓴다. - 내부에서 무슨 일이 일어나는지 더 투명하게 보임. <pre>for epoch in range(epochs): for x_batch, y_batch in dataloader: optimizer.zero_grad() # 이전 gradient 초기화 (직접 해야 함) y_pred = model(x_batch) # forward loss = loss_fn(y_pred, y_batch) loss.backward() # 역전파 optimizer.step() # 가중치 업데이트</pre>
- Keras는 "표준적인 학습 절차는 자동화하고, 예외적인 경우(GAN 등)만 손대게 해주는" 철학이고, PyTorch는 "학습 루프 자체를 항상 직접 쓰는 대신, 그만큼 뭐든 자유롭게 커스텀할 수 있는" 철학이다.	

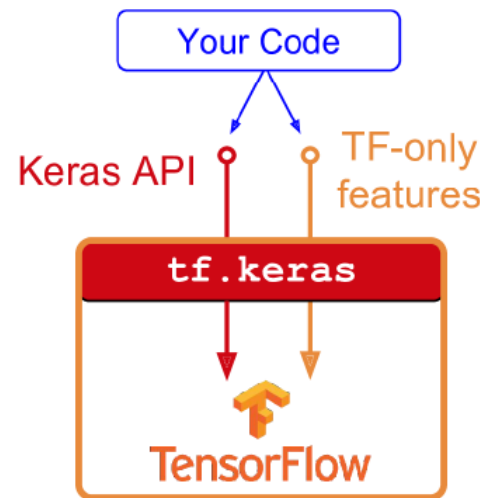
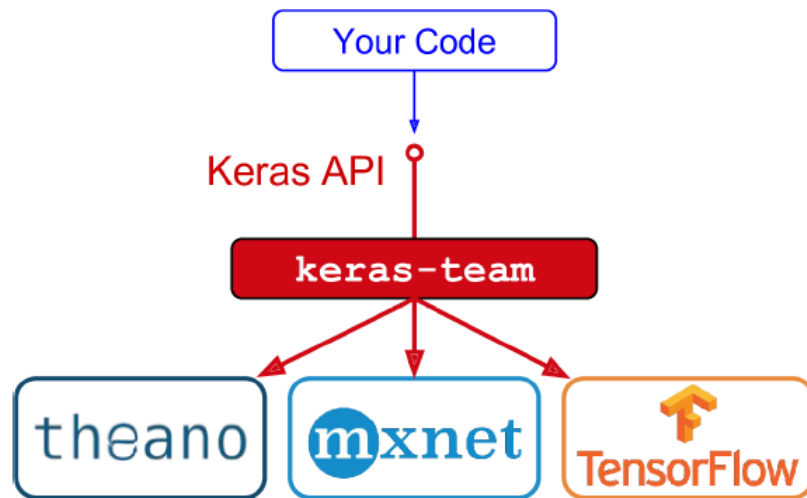
Deep Learning using Keras

Tensorflow and Keras

- **딥러닝 구현을 위한 도구: 텐서플로우와 케라스:**
 - 텐서플로우(TensorFlow): 구글이 개발한 가장 강력하고 대중적인 딥러닝 프레임워크다. 저수준(low-level)의 복잡한 연산부터 고수준(high-level) 모델링까지 모두 지원하는 핵심 엔진이다.
 - 케라스(Keras): 텐서플로우 위에서 작동하는 사용자 친화적인 고수준 API다. 마치 자동차의 '자동 변속기'처럼, 복잡한 내부 구현을 몰라도 쉽고 빠르게 딥러닝 모델을 만들 수 있게 해준다.
 - 우리는 Trensorflow 에 내장된 tf.keras 를 사용한다.

Tensorflow/Keras

- **Original Keras** (<https://keras.io>) by Francois Chollet
 - Can choose Backend from Tensorflow, Microsoft (CNTK), or Theano
- **Tensorflow Keras**
 - Tensorflow 2.0 comes bundled with its own Keras (tf.keras)
 - Only support Tensorflow as the backend



```
>>> import tensorflow as tf
>>> from tensorflow import keras
>>> tf.__version__
'2.0.0'
>>> keras.__version__
'2.2.4-tf'
```

Tensorflow/Keras

- History of Keras

시기	관계
~2019 (TF 2.0 발표)	"Keras를 TensorFlow의 공식 고수준 API로 채택" — Keras가 TF 안으로 흡수되어 <code>tf.keras</code> 로 통합됨. standalone <code>keras</code> 패키지는 사실상 <code>tf.keras</code> 로 리다이렉트
2019~2023	<code>tf.keras</code> 가 사실상 유일한 Keras. TensorFlow 없이는 Keras를 못 씀
2023 (Keras 3 발표)	다시 독립(decoupling) — Keras가 TF의 일부가 아니라, TF/JAX/PyTorch 중 뭐든 골라 쓸 수 있는 별도의 프레임워크로 재설계됨
TF 2.16~ (2024)	<code>tf.keras</code> 는 이제 "TF 안에서 Keras 3를 참조하는 통로"일 뿐이고, <code>import keras</code> 가 원래 자리를 되찾음

- 왜 "흡수됐다가 다시 독립"했나?

- 2019년 흡수의 목적: 당시엔 Keras 표준화 자체가 목표 (이때부터 [Keras = TF의 고수준 API](#) 로 확고하게 정리)
- 2023년 재독립의 목적: PyTorch/JAX 생태계가 커지면서, "TF에 묶여있지 않은 고수준 API"에 대한 수요가 생겼음. Keras 문법(model.fit(), Sequential(), Layer 등)은 그대로 두고, 그 밑단의 실제 텐서 연산(keras.ops)만 원하는 백엔드로 바꿔 끼울 수 있게 함.
- 즉, Keras가 TF의 전유물에서 벗어나, PyTorch/JAX 사용자도 원한다면 끌어올 수 있는 선택지가 됨. (Keras와 TensorFlow 모두 Google 에 속함 -> Keras라는 브랜드/생태계 자체의 영향력을 TF 바깥까지 넓히려는 전략으로 설명되고 있음)

Building a Model with Keras

- Sequential API

- Create the model

(모델 생성 방법 1: model.add() 사용)

```
model = keras.models.Sequential()  
model.add(keras.layers.Flatten(input_shape=[28, 28]))  
model.add(keras.layers.Dense(300, activation="relu"))  
model.add(keras.layers.Dense(100, activation="relu"))  
model.add(keras.layers.Dense(10, activation="softmax"))
```

(모델 생성 방법 2: 리스트로 한 번에 전달)

```
model = keras.models.Sequential([  
    keras.layers.Flatten(input_shape=[28, 28]),  
    keras.layers.Dense(300, activation="relu"),  
    keras.layers.Dense(100, activation="relu"),  
    keras.layers.Dense(10, activation="softmax")  
])
```

- Compile the Model (specify loss function and optimizer, [extra metrics])
- Training and evaluating
- Make prediction

Building a Model with Keras

- **Model compile**

- Compile() 메소드를 통해 학습 과정을 설정한다.

(*) sparse_categorical_crossentropy: when the target is integer (sparse) labeled (not one-hot encoded)

```
model.compile(loss="sparse_categorical_crossentropy", # 손실함수
              optimizer="sgd", # 최적화 알고리즘 (예: 'sgd', 'adam')
              metrics=["accuracy"]) # 훈련 중 모니터링할 지표 (예: 정확도)
```



Building a Model with Keras

- Model Train, Evaluate, and Predict

```
>>> history = model.fit(X_train, y_train, epochs=30,  
...                      validation_data=(X_valid, y_valid))
```

```
>>> model.evaluate(X_test, y_test)  
8832/10000 [=====] - ETA: 0s - loss: 0.4074 - acc: 0.8540  
[0.40738476498126985, 0.854]
```

```
>>> X_new = X_test[:3]  
>>> y_proba = model.predict(X_new)  
>>> y_proba.round(2)  
array([[0. , 0. , 0. , 0. , 0. , 0.09, 0. , 0.12, 0. , 0.79],  
       [0. , 0. , 0.94, 0. , 0.02, 0. , 0.04, 0. , 0. , 0. ],  
       [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],  
      dtype=float32)
```


구현의 이면: 행렬 연산

- 신경망의 모든 계산은 사실 행렬(Matrix) 연산으로 이루어진다.
- 입력 벡터 x 와 가중치 행렬 W 의 곱셈인 $H(x) = xW$ 로 표현.
- 수많은 데이터를 한 번에 처리하기 위해, 여러 데이터의 입력 벡터를 행으로 쌓은 행렬 X 를 만들어 가중치 행렬 W 와 곱셈한다. 이는 GPU를 이용한 병렬 처리에 매우 효율

WX vs XW

- Lecture (theory)

$$H(x) = Wx + b$$

$$h_{\theta}(x) = \theta_1 x + \theta_0$$
$$f(x) = ax + b$$

- Implementation (TensorFlow)

$$H(X) = XW$$

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n$$

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1 w_1 + x_2 w_2 + x_3 w_3)$$

$$H(X) = XW$$

구현의 이면: 행렬 연산

x_1	x_2	x_3	y
73	80	75	152
93	88	93	185
89	91	90	180
96	98	100	196
73	66	70	142

Test Scores for General Psychology

$$H(x_1, x_2, x_3) = w_1 x_1 + w_2 x_2 + w_3 x_3$$

$$(x_1 \ x_2 \ x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1 w_1 + x_2 w_2 + x_3 w_3)$$

$$H(X) = XW$$



$$w_1 x_1 + w_2 x_2 + w_3 x_3 + \dots + w_n x_n$$

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_{11} w_1 + x_{12} w_2 + x_{13} w_3 \\ x_{21} w_1 + x_{22} w_2 + x_{23} w_3 \\ x_{31} w_1 + x_{32} w_2 + x_{33} w_3 \\ x_{41} w_1 + x_{42} w_2 + x_{43} w_3 \\ x_{51} w_1 + x_{52} w_2 + x_{53} w_3 \end{pmatrix}$$

$$H(X) = XW$$

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_{11} w_1 + x_{12} w_2 + x_{13} w_3 \\ x_{21} w_1 + x_{22} w_2 + x_{23} w_3 \\ x_{31} w_1 + x_{32} w_2 + x_{33} w_3 \\ x_{41} w_1 + x_{42} w_2 + x_{43} w_3 \\ x_{51} w_1 + x_{52} w_2 + x_{53} w_3 \end{pmatrix}$$

$[n, 3]$

$[3, 1]$

$[n, \overset{1}{1}]$

$$H(X) = XW$$

Hypothesis using Matrices

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \text{?} = \begin{pmatrix} x_{11}w_{11} + x_{12}w_{21} + x_{13}w_{31} & x_{11}w_{12} + x_{12}w_{22} + x_{13}w_{32} \\ x_{21}w_{11} + x_{22}w_{21} + x_{23}w_{31} & x_{21}w_{12} + x_{22}w_{22} + x_{23}w_{32} \\ x_{31}w_{11} + x_{32}w_{21} + x_{33}w_{31} & x_{31}w_{12} + x_{32}w_{22} + x_{33}w_{32} \\ x_{41}w_{11} + x_{42}w_{21} + x_{43}w_{31} & x_{41}w_{12} + x_{42}w_{22} + x_{43}w_{32} \\ x_{51}w_{11} + x_{52}w_{21} + x_{53}w_{31} & x_{51}w_{12} + x_{52}w_{22} + x_{53}w_{32} \end{pmatrix}$$

$[n, 3]$

$[?, ?]$

$[n, 2]$

$$H(X) = XW$$

Hypothesis using Matrices (Example)

- Easier and simpler notation (Using Matrices)

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1 w_1 + x_2 w_2 + x_3 w_3)$$

$$H(X) = XW$$

```
# initialize W
w1 = tf.Variable(tf.random_normal([1]))
w2 = tf.Variable(tf.random_normal([1]))
w3 = tf.Variable(tf.random_normal([1]))
```

```
# hypothesis, prediction function
w1 * x1 + w2 * x2 + w3 * x3 + b
```

```
# update w1, w2, w3
w1.assign_sub(learning_rate * w1_grad)
w2.assign_sub(learning_rate * w2_grad)
w3.assign_sub(learning_rate * w3_grad)
```

```
# initialize W
W = tf.Variable(tf.random_normal([3, 1]))
```

```
# hypothesis, prediction function
tf.matmul(X, W) + b
```

```
# updates parameters (W and b)
W.assign_sub(learning_rate * W_grad)
```

Hypothesis using Matrices (Example)

- Low-level Tensorflow 구현 (Keras의 .fit() 메소드 내부)

$$H(X) = XW$$

```
data = np.array([
    # X1,    X2,    X3,    y
    [ 73.,   80.,   75.,  152. ],
    [ 93.,   88.,   93.,  185. ],
    [ 89.,   91.,   90.,  180. ],
    [ 96.,   98.,  100.,  196. ],
    [ 73.,   66.,   70.,  142. ]
], dtype=np.float32)

# slice data
X = data[:, :-1]
y = data[:, [-1]]
```

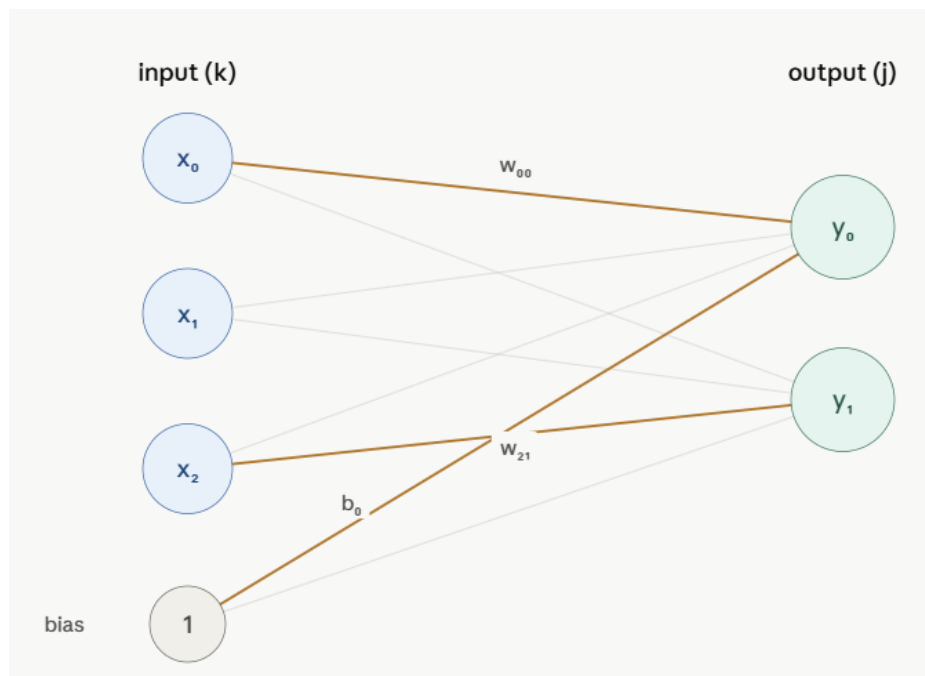
```
# 가중치 W와 편향 b를 변수로 직접 선언
W = tf.Variable(tf.random.normal([3, 1]))
b = tf.Variable(tf.random.normal([1]))

# 학습 루프
for i in range(2001):
    # GradientTape로 그래디언트를 기록
    with tf.GradientTape() as tape:
        # 가설(예측)과 비용(손실) 계산
        hypothesis = tf.matmul(X, W) + b
        cost = tf.reduce_mean(tf.square(hypothesis - y))

    # 그래디언트 계산 및 가중치 업데이트
    W_grad, b_grad = tape.gradient(cost, [W, b])
    W.assign_sub(learning_rate * W_grad)
    b.assign_sub(learning_rate * b_grad)
```

Matrix Multiplication (Example)

- (예) 순방향 전파 (Forward): 입력 3, 출력 2, Batch size=4 인 경우



$$X_{(4 \times 3)} = \begin{bmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \\ x_{30} & x_{31} & x_{32} \end{bmatrix} \quad W_{(3 \times 2)} = \begin{bmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \\ w_{20} & w_{21} \end{bmatrix} \quad b_{(2,)} = [b_0 \quad b_1]$$

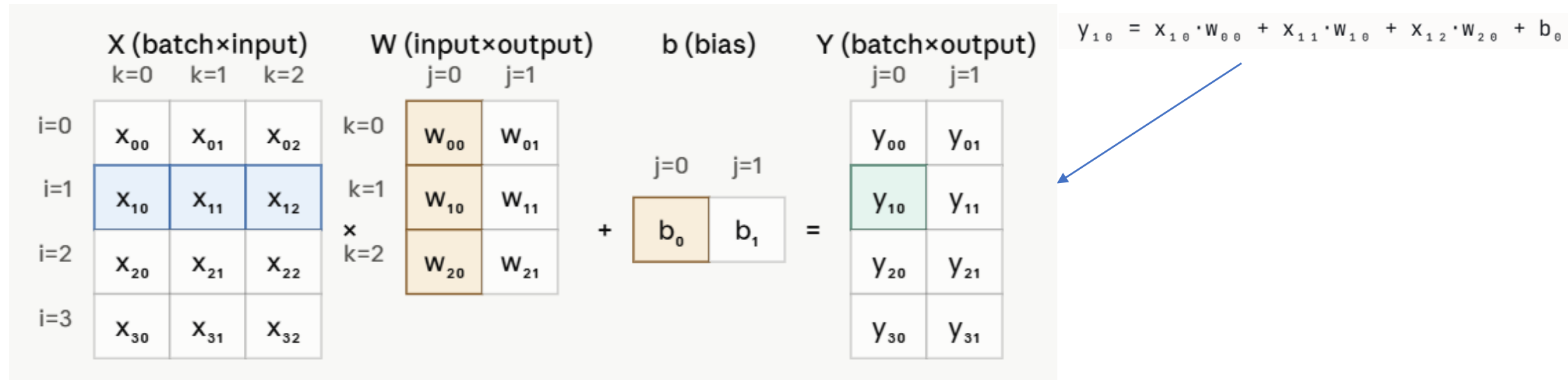
$$Y = XW + b$$

$$Y = \begin{bmatrix} x_{00}w_{00} + x_{01}w_{10} + x_{02}w_{20} + b_0 & x_{00}w_{01} + x_{01}w_{11} + x_{02}w_{21} + b_1 \\ x_{10}w_{00} + x_{11}w_{10} + x_{12}w_{20} + b_0 & x_{10}w_{01} + x_{11}w_{11} + x_{12}w_{21} + b_1 \\ x_{20}w_{00} + x_{21}w_{10} + x_{22}w_{20} + b_0 & x_{20}w_{01} + x_{21}w_{11} + x_{22}w_{21} + b_1 \\ x_{30}w_{00} + x_{31}w_{10} + x_{32}w_{20} + b_0 & x_{30}w_{01} + x_{31}w_{11} + x_{32}w_{21} + b_1 \end{bmatrix}$$

$$y_{i,j} = \sum_{k=0}^2 x_{i,k} \cdot w_{k,j} + b_j$$

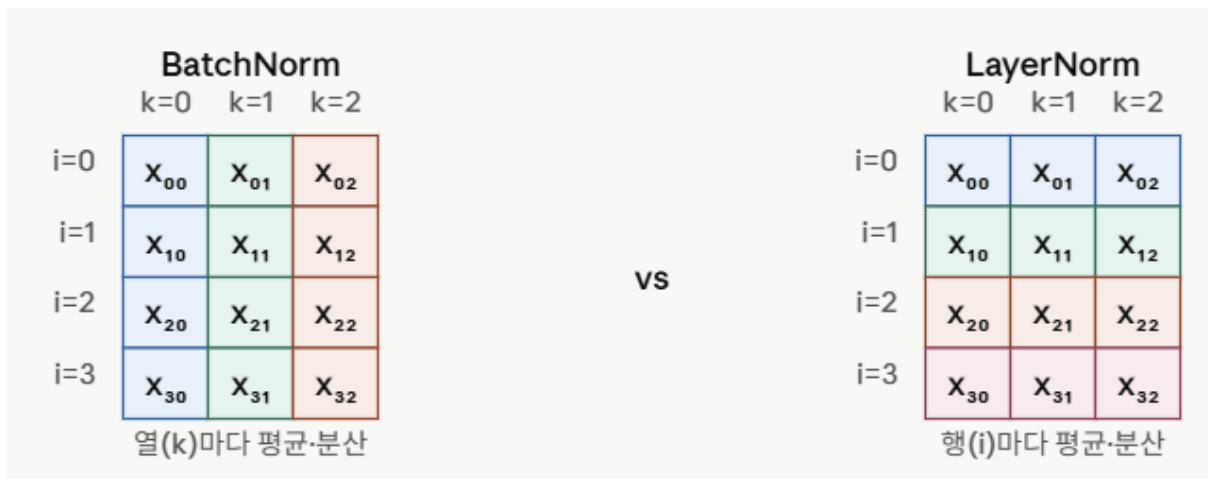
Matrix Multiplication (Example)

- (예) 순방향 전파 (Forward): 입력 3, 출력 2, Batch size=4 인 경우



- X 의 shape: (batch_size=4, input_dim=3) — 행(i)은 "몇 번째 샘플인가", 열(k)은 "몇 번째 입력 특징인가"
- W 의 shape: (input_dim=3, output_dim=2) — 행(k)은 input 방향, 열(j)은 output 방향. 즉 $w_{k,j}$ 는 "k번째 입력이 j번째 출력에 얼마나 기여하는가"를 나타내는 가중치입니다.
- b 의 shape: (output_dim=2,) — 모든 샘플(각 행)에 동일하게 broadcast되어 더해집니다.
- Y 의 shape: (batch_size=4, output_dim=2)

Normalization: BatchNorm and LayerNorm



- **BatchNorm** : 열(k)마다, 즉 배치(i) 방향으로 계산
 - 같은 특징(feature) k에 대해, 배치 안의 모든 샘플 (i=0~3)을 모아서 평균/분산을 낸다.
 - 배치 크기가 커야 통계가 안정적

$$\mu_k = \frac{1}{4} \sum_{i=0}^3 x_{i,k} \quad \sigma_k^2 = \frac{1}{4} \sum_{i=0}^3 (x_{i,k} - \mu_k)^2$$

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_k}{\sqrt{\sigma_k^2 + \epsilon}} \quad y_{i,k} = \gamma_k \hat{x}_{i,k} + \beta_k$$

- **LayerNorm** : 행(i)마다, 즉 특징(k) 방향으로 계산
 - 같은 샘플 i에 대해, 그 샘플의 모든 특징(k=0~2)을 모아서 평균/분산을 낸다. (다른 샘플이 몇 개 있는지와 무관하게, 샘플 하나만으로 정규화)

$$\mu_i = \frac{1}{3} \sum_{k=0}^2 x_{i,k} \quad \sigma_i^2 = \frac{1}{3} \sum_{k=0}^2 (x_{i,k} - \mu_i)^2$$

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \quad y_{i,k} = \gamma_k \hat{x}_{i,k} + \beta_k$$

Why Normalization?

- **Why Normalization?**
 - 레이어를 거칠 때마다 값의 분포가 계속 흔들리는 걸 막아서 [학습을 안정시키기 위해서임](#).
- **정규화가 없으면 생기는 문제:** 레이어를 여러 개 쌓으면, 앞쪽 레이어의 가중치가 학습 중 조금만 바뀌어도 그 출력값의 스케일(평균, 분산)이 요동친다. 그러면 뒤쪽 레이어는 매 스텝마다 계속 달라지는 입력 분포에 적응해야 해서:
 - 학습률을 조금만 높여도 값이 발산하기 쉬워짐
 - 깊은 네트워크일수록 이 효과가 누적돼서 학습이 아예 안 되는 경우도 생김
 - 가중치 초기화 방식에 결과가 지나치게 민감해짐
- **정규화를 하면:** 각 레이어의 출력을 평균 0, 분산 1 근처로 강제로 맞춰주면, 뒤쪽 레이어는 항상 비슷한 스케일의 입력을 받게 되어:
 - 더 큰 학습률을 써도 안정적으로 학습됨 (실제로 학습 속도가 빨라짐)
 - 초기화에 덜 민감해짐
 - γ, β (scale, shift) 파라미터를 따로 학습시켜서, "정규화는 하되 필요하면 다시 원래 스케일로 조정할 자유"도 남겨둠 — 그래서 정규화가 표현력을 뺏지는 않습니다.

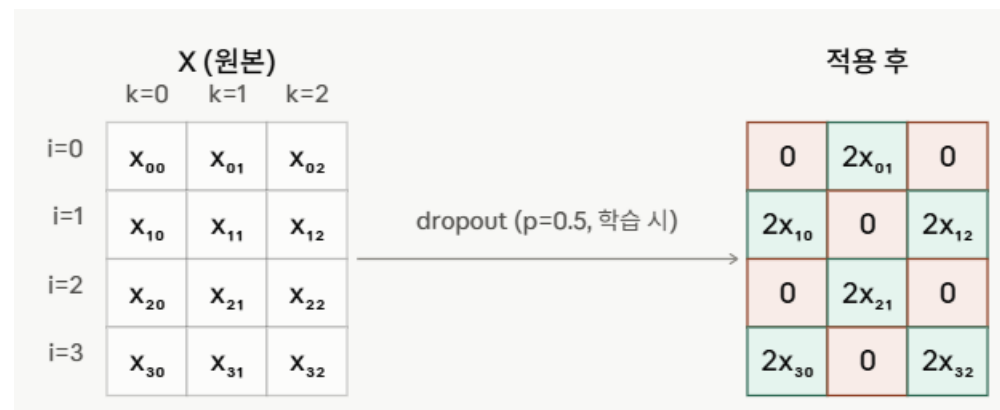
BatchNorm and LayerNorm

	BatchNorm	LayerNorm
정규화 계산 축	배치(i) 방향 — 같은 feature, 여러 샘플	feature(k) 방향 — 한 샘플 안의 여러 feature
배치 크기 민감도	민감함 — 배치가 작으면 통계 불안정, batch_size=1이면 분산=0으로 붕괴	무관함 — batch_size=1이어도 정상 동작
학습 vs 추론 동작	다름 — 학습 땀 그 배치의 실제 통계, 추론 땀 저장해둔 이동평균(running mean/var) 사용	같음 — 항상 그 샘플만 보고 계산, 저장할 통계 자체가 없음
배치 내 샘플 간 상호의존	있음 — 같은 배치의 다른 샘플에 따라 결과가 미세하게 달라짐	없음 — 샘플마다 완전히 독립적으로 계산
적합한 상황	배치 크기가 크고 고정적인 경우 (CNN, 이미지)	배치 크기가 작거나 가변적, 시퀀스 길이가 매번 다른 경우 (RNN, Transformer)
대표 사례	GAN의 generator	Transformer, LLM 계열 대부분

- 넣는 위치는 "가중치가 있는 레이어(Dense, Conv) → Normalization → 활성화 함수" 순서로 한다. 즉, 학습 가능한 파라미터가 곱해지는 레이어 뒤에 붙여서, 그 출력이 활성화 함수에 들어가기 전에 스케일을 정리해주는 역할을 한다.
- 레이어를 깊게 쌓을수록, 그리고 그 레이어가 학습 가능한 가중치를 갖고 있을수록" 정규화가 필요해지는 경향이 있고, 마지막 출력 레이어처럼 "이제 진짜 값이 나와야 하는 지점"에는 보통 안 넣는다는 게 보통이다.

Drop Out

- 일부 원소를 확률적으로 0으로 만들고, 남은 값은 스케일을 키워서 보정하는 것.
- 예: $p=0.5$ (50% 확률로 끔)
 - 살아남은 원소는 그냥 두는 게 아니라 $1/(1-p)$ 로 곱해서 값을 키운다.
- 학습(training) 때만 적용된다는 게 핵심
 - 학습 때: 매 배치마다 무작위로 다른 마스크를 뽑아서 일부를 꺼버림 (그림처럼)
 - 추론(predict()) 때: 마스크 없이 전부 그대로 통과 - 대신 위에서 이미 학습 때 스케일을 보정해 뒀기 때문에, 추론 때 따로 스케일을 조정할 필요가 없다.

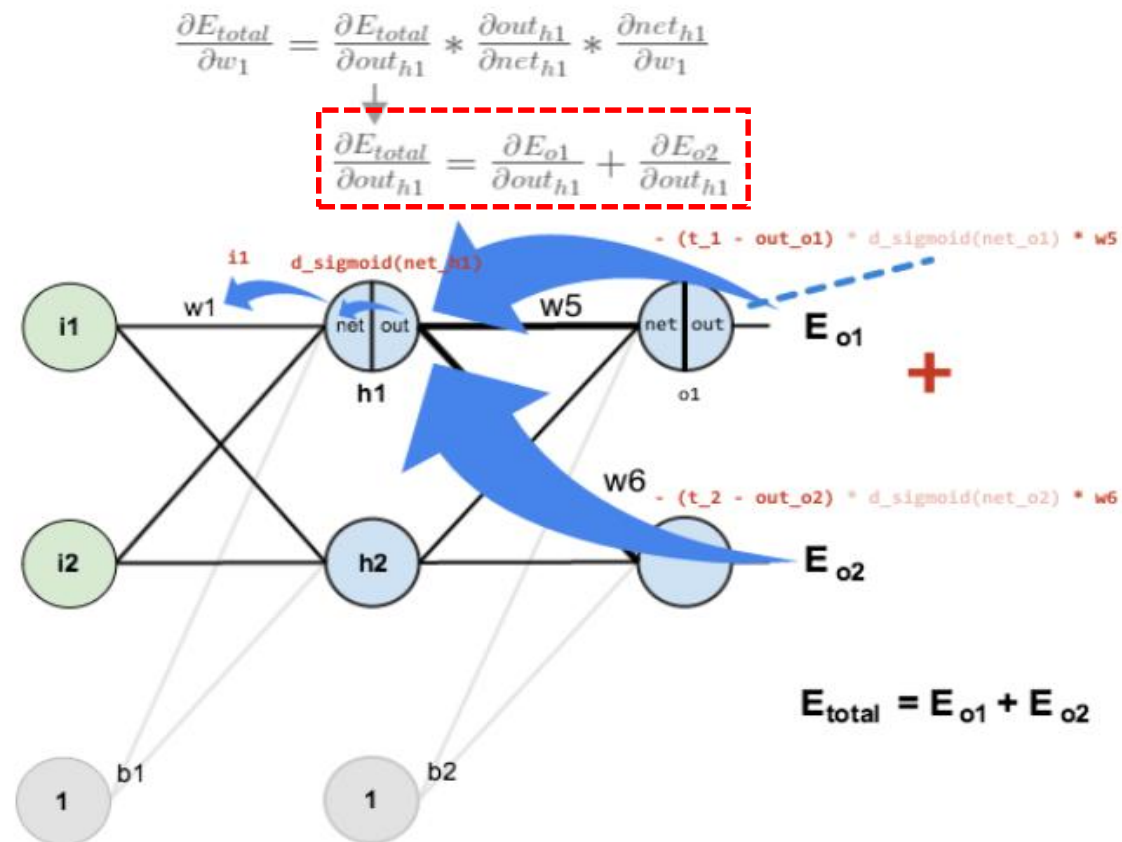
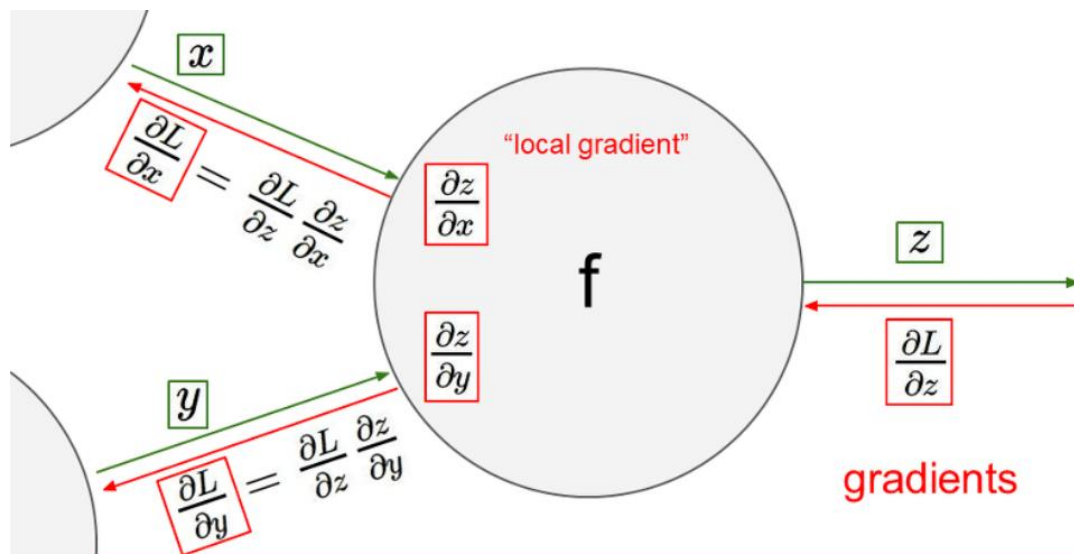


- 왜 이렇게 하나 (효과)
 - 매 배치마다 무작위로 다른 뉴런 조합이 꺼지니, 네트워크가 "특정 뉴런 몇 개에만 과하게 의존하는 것"을 막아준다. (과대적합 방지 효과)
 - 매번 다른 부분집합으로 같은 문제를 풀도록 강제하는 셈이라, 여러 개의 작은 네트워크를 앙상블하는 것과 비슷한 효과를 낸다.

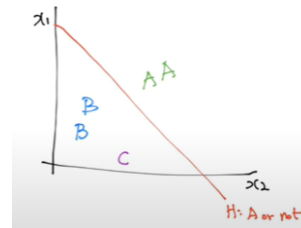
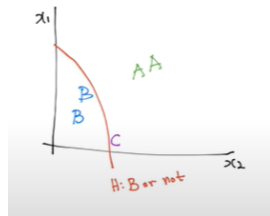
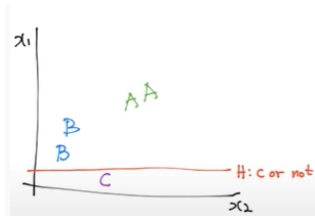
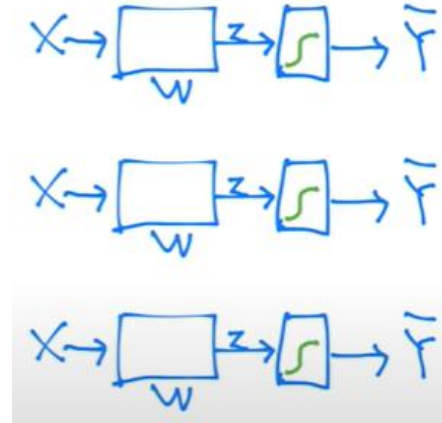
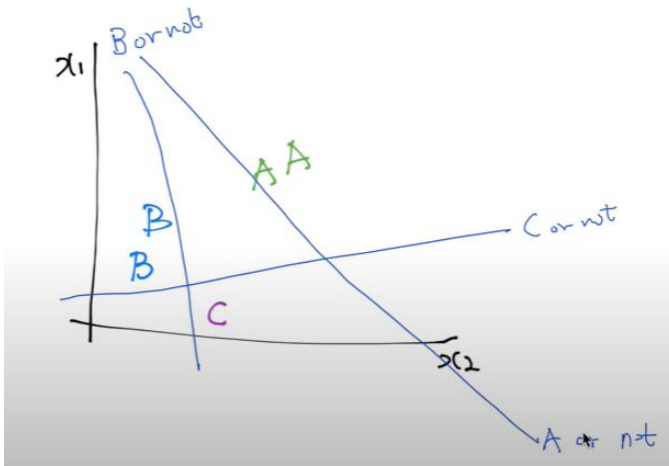
여기까지

Back-Propagation

- Use Chain-rule



Multinomial (Multi-class) Classification



$$\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1 x_1 + w_2 x_2 + w_3 x_3]$$

$$\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1 x_1 + w_2 x_2 + w_3 x_3]$$

$$\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = [w_1 x_1 + w_2 x_2 + w_3 x_3]$$

Multinomial (Multi-class) Classification

$$\begin{bmatrix} w_{A1} & w_{A2} & w_{A3} \\ w_{B1} & w_{B2} & w_{B3} \\ w_{C1} & w_{C2} & w_{C3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} w_{A1}x_1 + w_{A2}x_2 + w_{A3}x_3 \\ w_{B1}x_1 + w_{B2}x_2 + w_{B3}x_3 \\ w_{C1}x_1 + w_{C2}x_2 + w_{C3}x_3 \end{bmatrix} = \begin{bmatrix} \bar{y}_A \\ \bar{y}_B \\ \bar{y}_C \end{bmatrix} \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix}$$

- Softmax function

$$Y \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \rightarrow \begin{bmatrix} 0.7 \\ 0.2 \\ 0.1 \end{bmatrix}$$

$$S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

SCORES $\longrightarrow$ PROBABILITIES

Multinomial (Multi-class) Classification

- Loss function for Multinomial classification

CROSS-ENTROPY

$S(Y)$

L

$D(S, L) = -\sum_i L_i \log(S_i)$

<https://www.udacity.com/course/viewer#!/c-ud730/l-6370362152/m-6379811817>

$C(H(x), y) = y \log(H(x)) - (1 - y) \log(1 - H(x))$

Logistic cost (binary cross-entropy)

$D(S, L) = -\sum_i L_i \log(S_i)$

Cross-entropy cost

Multinomial (Multi-class) Classification

- Cross Entropy (교차 엔트로피)

$$E = -\sum_k t_k \log y_k$$

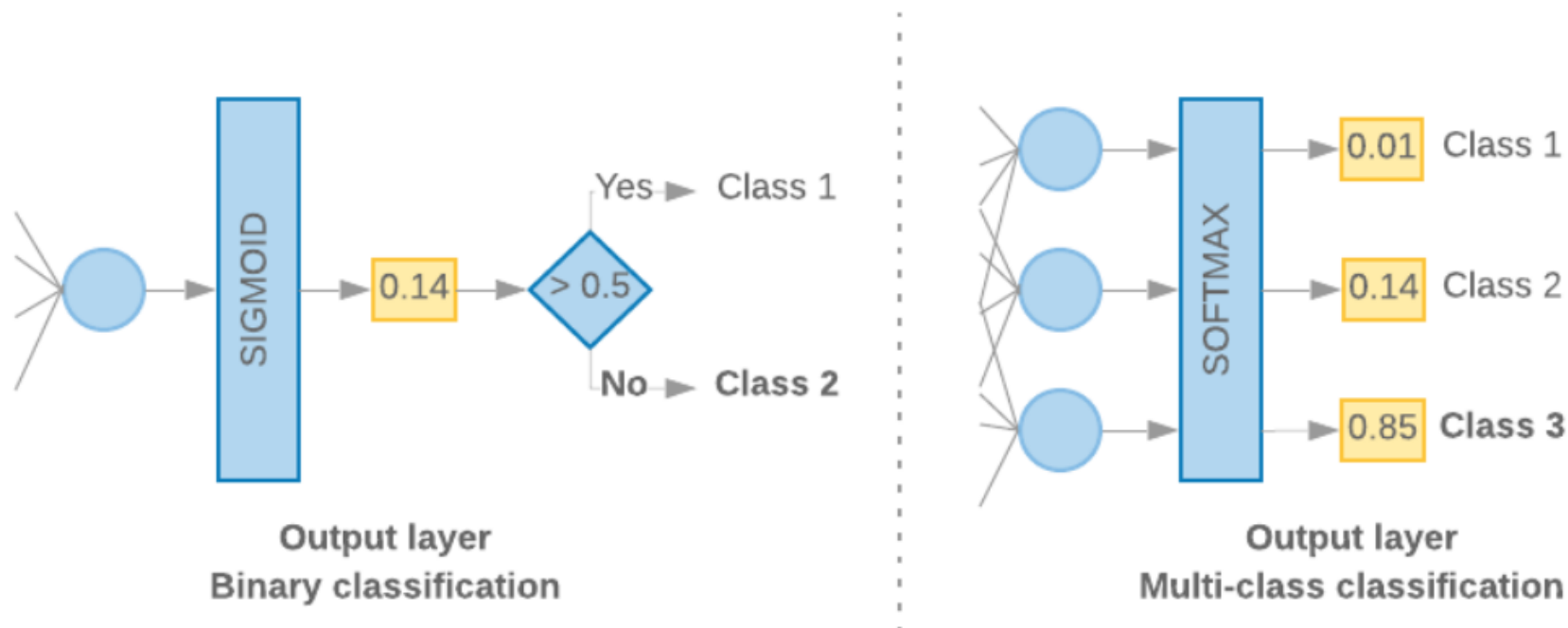
(y: predict, t: target, k: dimension, log: natural log)

- Softmax (소프트맥스)

$$\sigma(j) = \frac{\exp(\mathbf{w}_j^\top \mathbf{x})}{\sum_{k=1}^K \exp(\mathbf{w}_k^\top \mathbf{x})} = \frac{\exp(z_j)}{\sum_{k=1}^K \exp(z_k)}$$

Cross Entropy - Softmax

- Output layer of the Classifier in Deep Learning



Cross Entropy - Softmax

- 상대적인 점수 비교 : 확률처럼 0~1 사이 값으로 매핑

